POLIFY

Progetto di Tecnologie Informatiche per il Web
a.a. 2024/2025

Traccia numero 2: Playlist Musicale

D’Ambrosi Giuseppe (10848505)

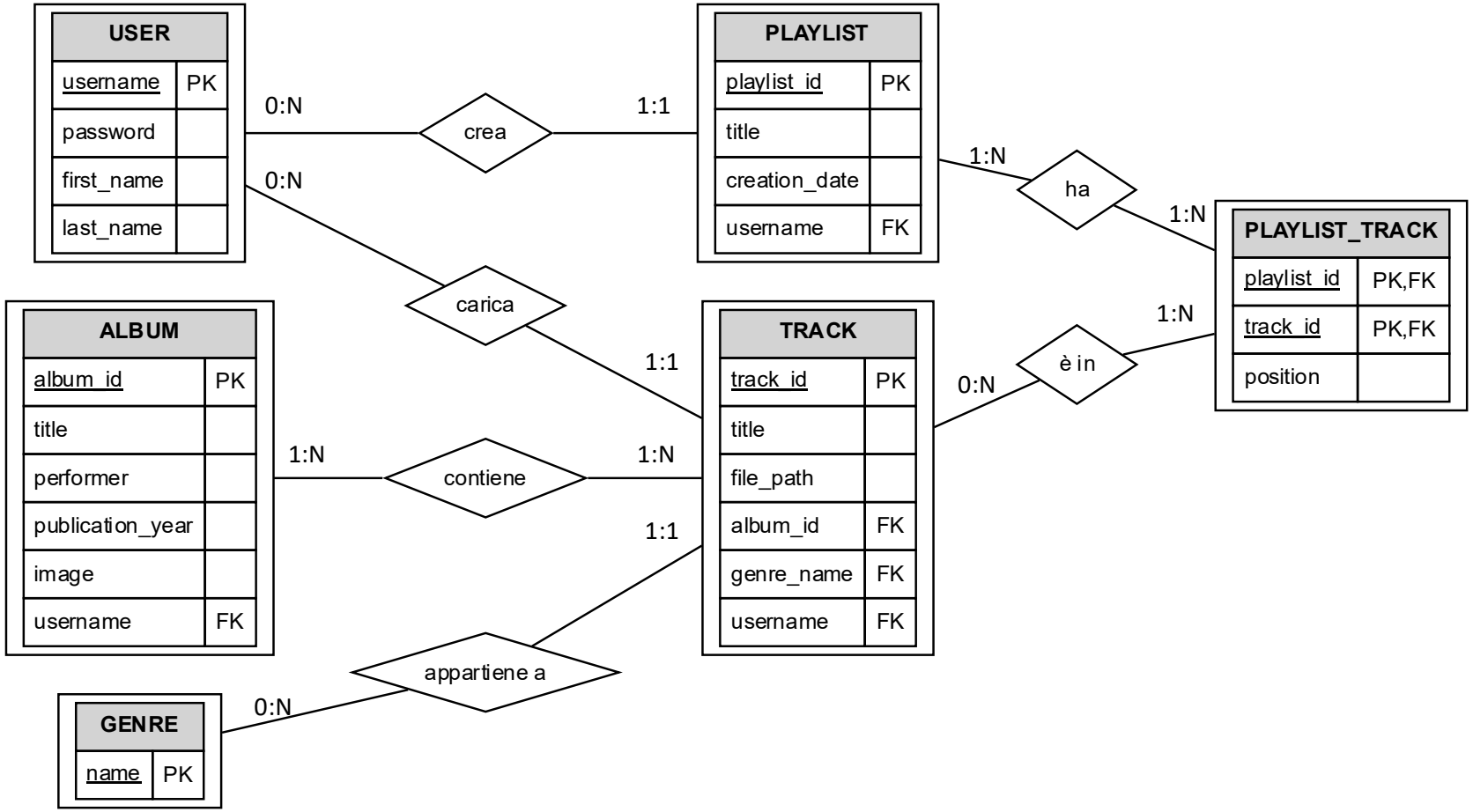
Di Clerico Giacomo (10838815)

PAGINE, COMPONENTI VISIVI, EVENTI, AZIONI

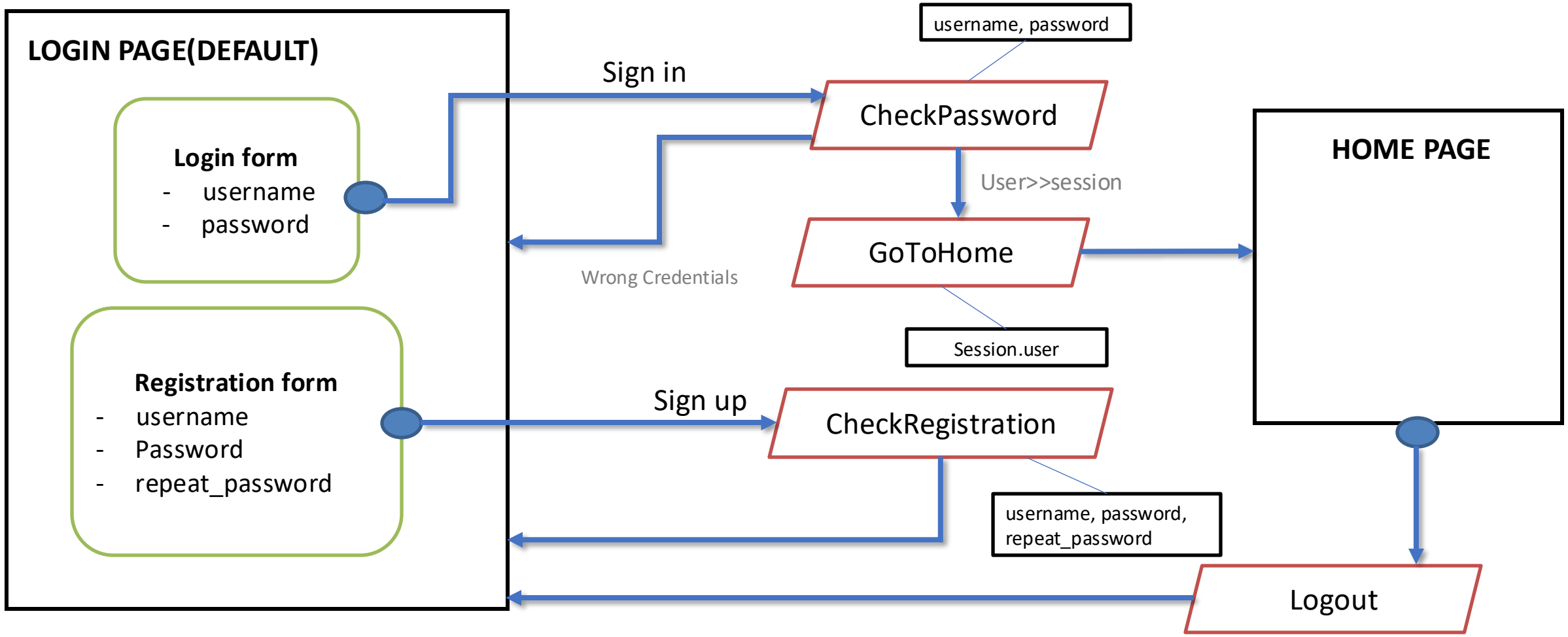
Specifica: versione pure HTML

Un'applicazione web consente la gestione di una playlist di brani musicali. Playlist e brani sono personali di ogni utente e non condivisi. Ogni utente ha username, password, nome e cognome. Ogni brano musicale è memorizzato nella base di dati mediante un titolo, l'immagine e il titolo dell'album da cui il brano è tratto, il nome dell'interprete (singolo o gruppo) dell'album, l'anno di pubblicazione dell'album, il genere musicale (si supponga che i generi siano prefissati) e il file musicale. Non è richiesto di memorizzare l'ordine con cui i brani compaiono nell'album a cui appartengono. Si ipotizzi che un brano possa appartenere a un solo album (no compilation). L'utente, previo login, può creare brani mediante il caricamento dei dati relativi e raggrupparli in playlist. Una playlist è un insieme di brani scelti tra quelli caricati dallo stesso utente. Lo stesso brano può essere inserito in più playlist. Una playlist ha un titolo e una data di creazione ed è associata al suo creatore. A seguito del login, l'utente accede all'HOME PAGE che presenta l'elenco delle proprie playlist, ordinate per data di creazione decrescente, un form per caricare un brano con tutti i dati relativi e un form per creare una nuova playlist. Il form per la creazione di una nuova playlist mostra l'elenco dei brani dell'utente ordinati per ordine alfabetico crescente dell'autore o gruppo e per data crescente di pubblicazione dell'abum a cui il brano appartiene. Tramite il form è possibile selezionare uno o più brani da includere. Quando l'utente clicca su una playlist nell'HOME PAGE, appare la pagina PLAYLIST PAGE che contiene inizialmente una tabella di una riga e cinque colonne. Ogni cella contiene il titolo di un brano e l'immagine dell'album da cui proviene. I brani sono ordinati da sinistra a destra per ordine alfabetico crescente dell'autore o gruppo e per data crescente di pubblicazione dell'abum a cui il brano appartiene. Se la playlist contiene più di cinque brani, sono disponibili comandi per vedere il precedente e successivo gruppo di brani. Se la pagina PLAYLIST mostra il primo gruppo e ne esistono altri successivi nell'ordinamento, compare a destra della riga il bottone SUCCESSIVI, che permette di vedere il gruppo successivo. Se la pagina PLAYLIST mostra l'ultimo gruppo e ne esistono altri precedenti nell'ordinamento, compare a sinistra della riga il bottone PRECEDENTI, che permette di vedere i cinque brani precedenti. Se la pagina PLAYLIST mostra un blocco e esistono sia precedenti sia successivi, compare a destra della riga il bottone SUCCESSIVI e a sinistra il bottone PRECEDENTI. La pagina PLAYLIST contiene anche un form che consente di selezionare e aggiungere uno o più brani alla playlist corrente, se non già presente nella playlist. Tale form presenta i brani da scegliere nello stesso modo del form usato per creare una playlist. A seguito dell'aggiunta di un brano alla playlist corrente, l'applicazione visualizza nuovamente la pagina a partire dal primo blocco della playlist. Quando l'utente seleziona il titolo di un brano, la pagina PLAYER mostra tutti i dati del brano scelto e il player audio per la riproduzione del brano.

Design del database - ER

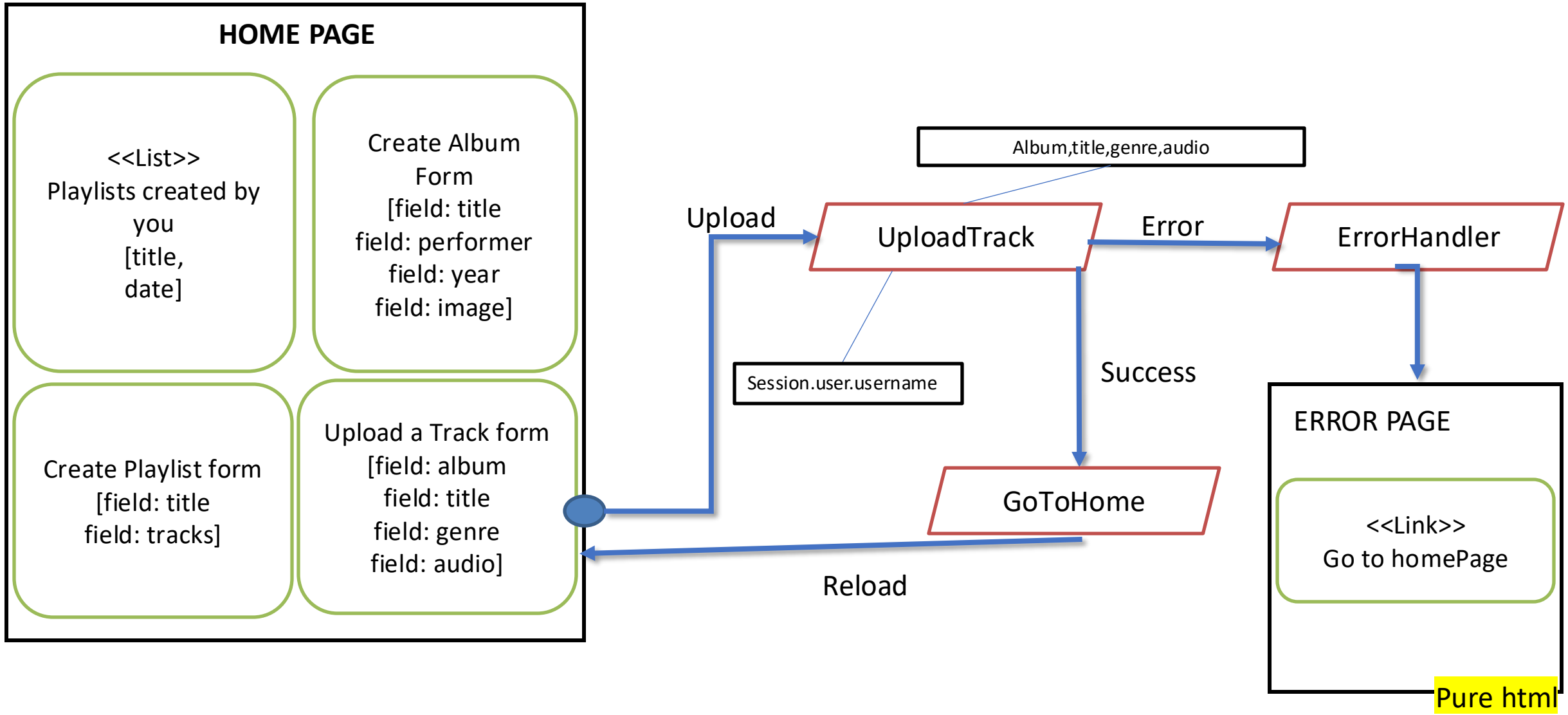


Application design

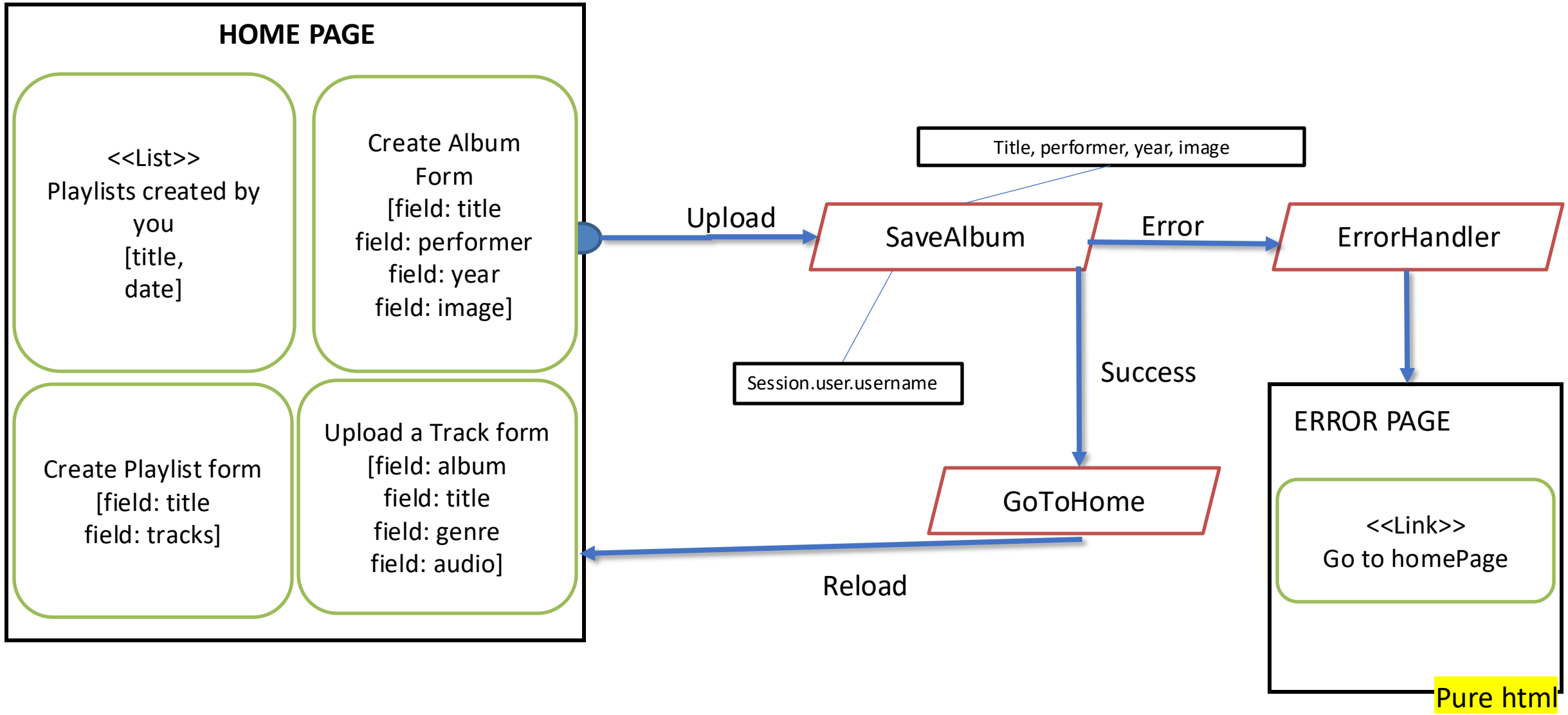


Pure html

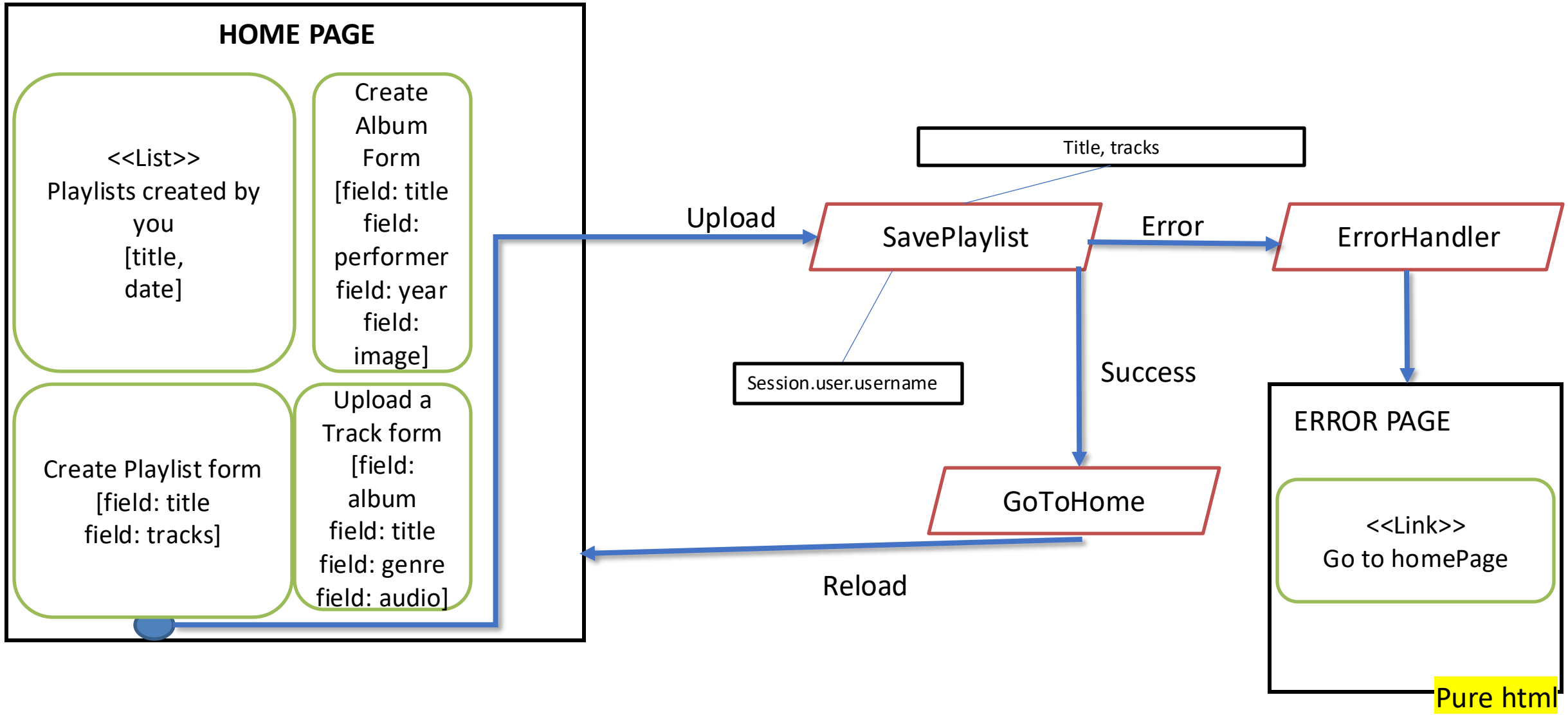
Application design



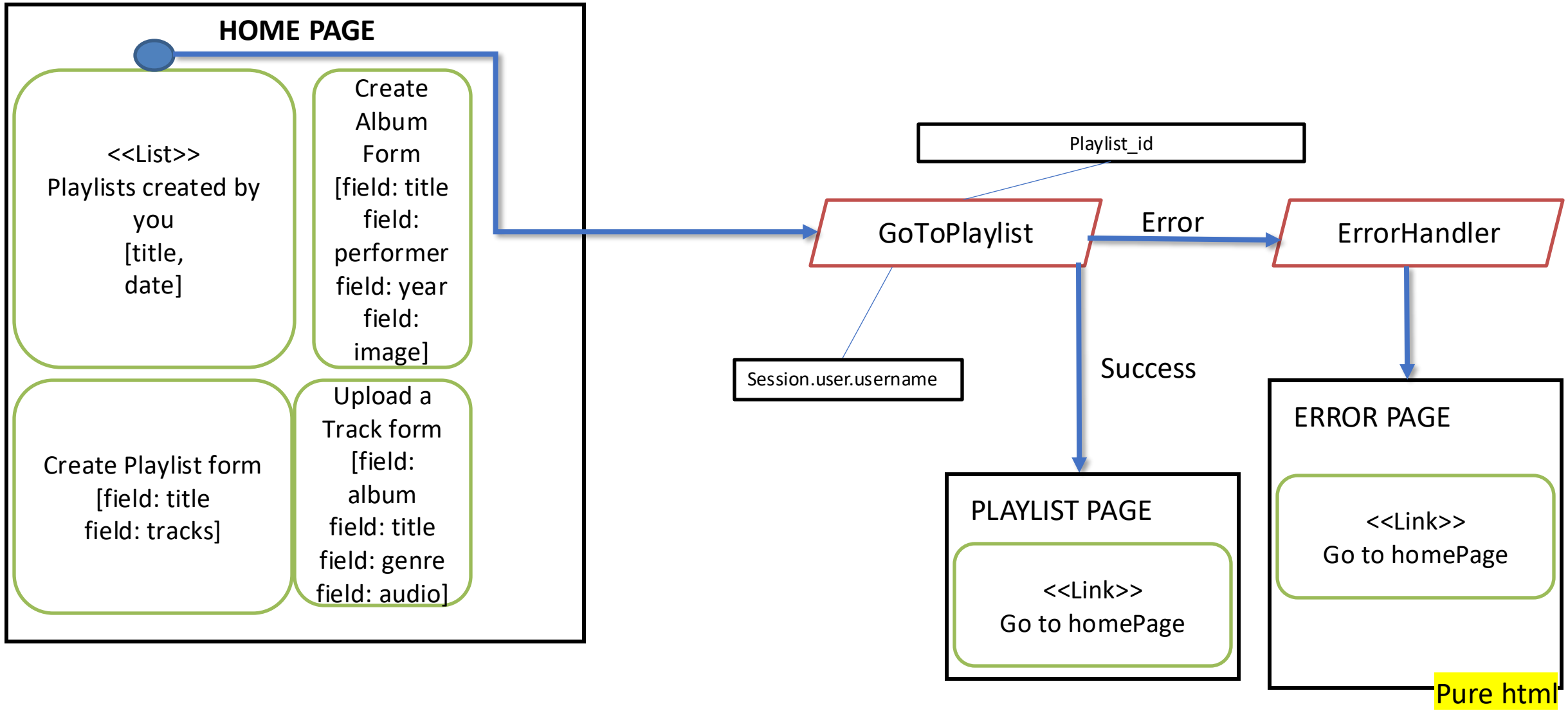
Application design



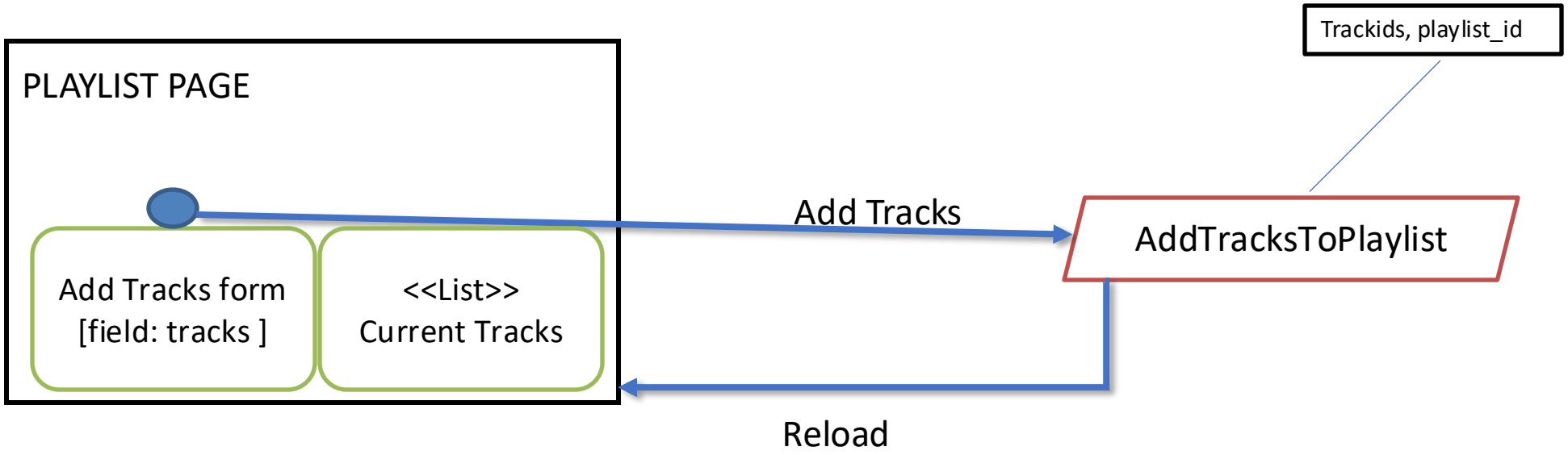
Application design



Application design

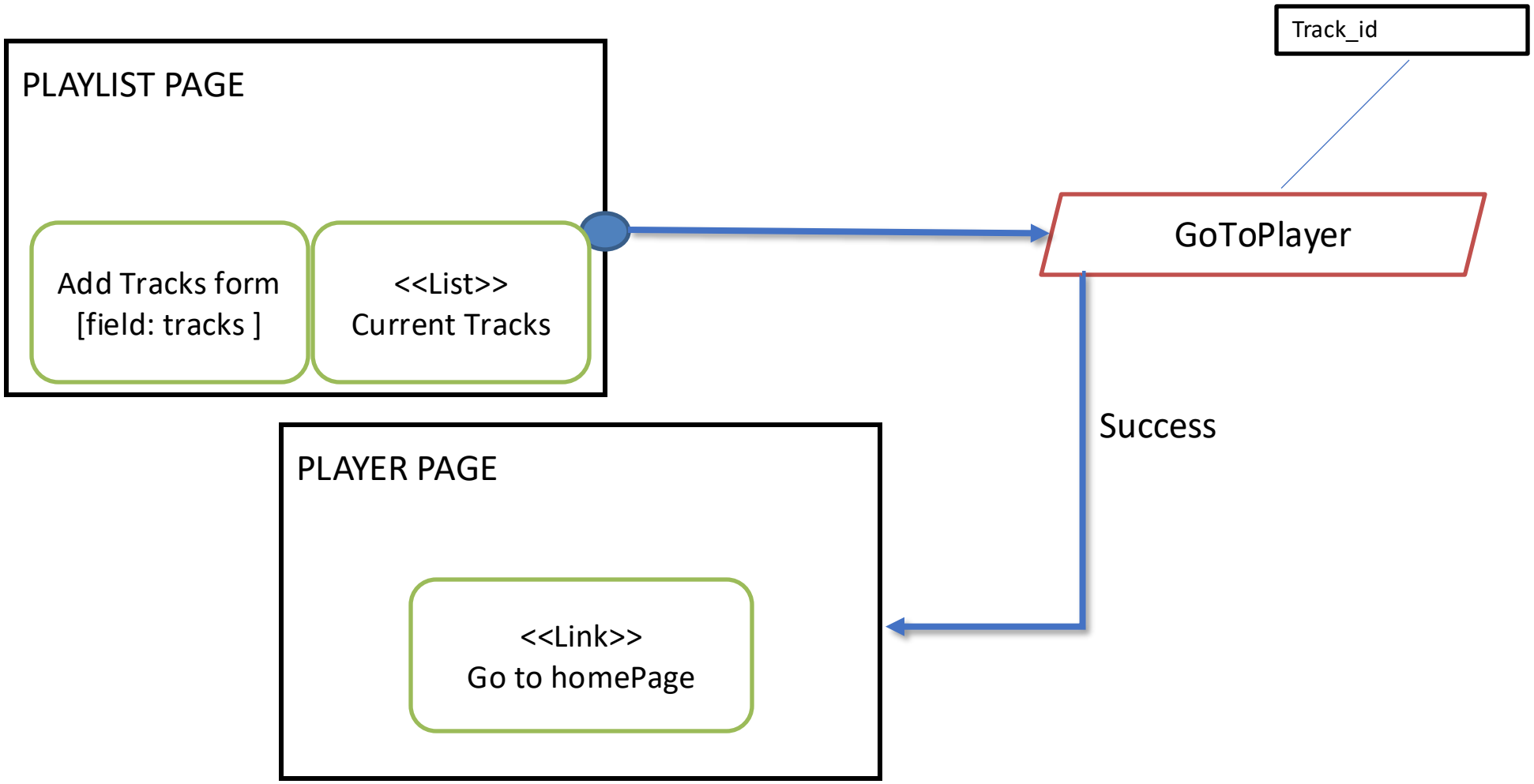


Application design



Pure html

Application design



Pure html

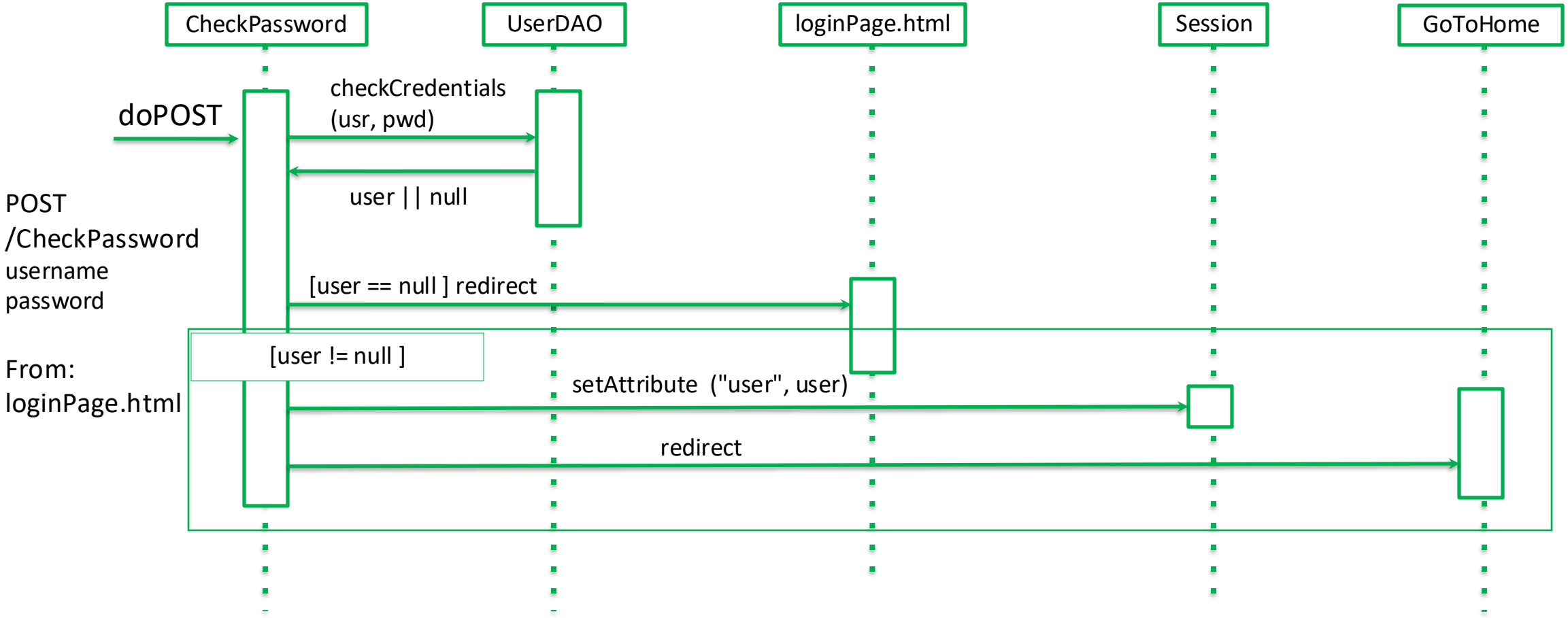
Componenti

- **DAO**
  - **AlbumDAO**
    create(*title*, *performer*, *publicationyear*, *path*, *username*) –> void
    getAlbumOfUser(*username*) -> List<Album>
  - **GenresDAO**
    getGenres() -> List<Genres>
    getGenresNames() ->List<String>
  - **PlaylistDAO**
    getPlaylistsOfUser(*username*) -> List<Playlist>
    createPlaylistWithTracks(*title*, *username*, <List> *trackIds*) -> void
    getPlaylistById(*playlistId*) -> Playlist
    addTrackToPlaylist(*playlistId*, *trackId*) -> void
    isOwnedBy(*playlistId*, *username*) -> boolean
  - **TrackDAO**
    create(*title*, *filePath*, *albumId*, *genreName*, *username*) -> void
    findByUserOrdered(*username*) -> List <Track>
    findById(*trackId*) -> Track
    getTracksByPlaylistOrdered(*playlistId*) -> List <Track>
    isOwnedBy(*trackId*, *username*) -> Boolean
  - **UserDAO**
    checkCredentials(*user*, *pwd*) -> User
    registerUser(*user*) –> void
    findAllUsers() -> ArrayList<User>

- **Servlets**
  - CheckPassword
  - CheckRegistration
  - Logout
  - GoToHome
  - AddTracksToPlaylist
  - ErrorHandler
  - GoToPlayer
  - GoToPlaylist
  - SaveAlbum
  - SavePlaylist
  - UploadTrack
- **Templates**
  - loginPage
  - homePage
  - playlistPage
  - playerPage
  - errorPage
- **Beans**
  - Album
  - Genres
  - Playlist
  - Track
  - User

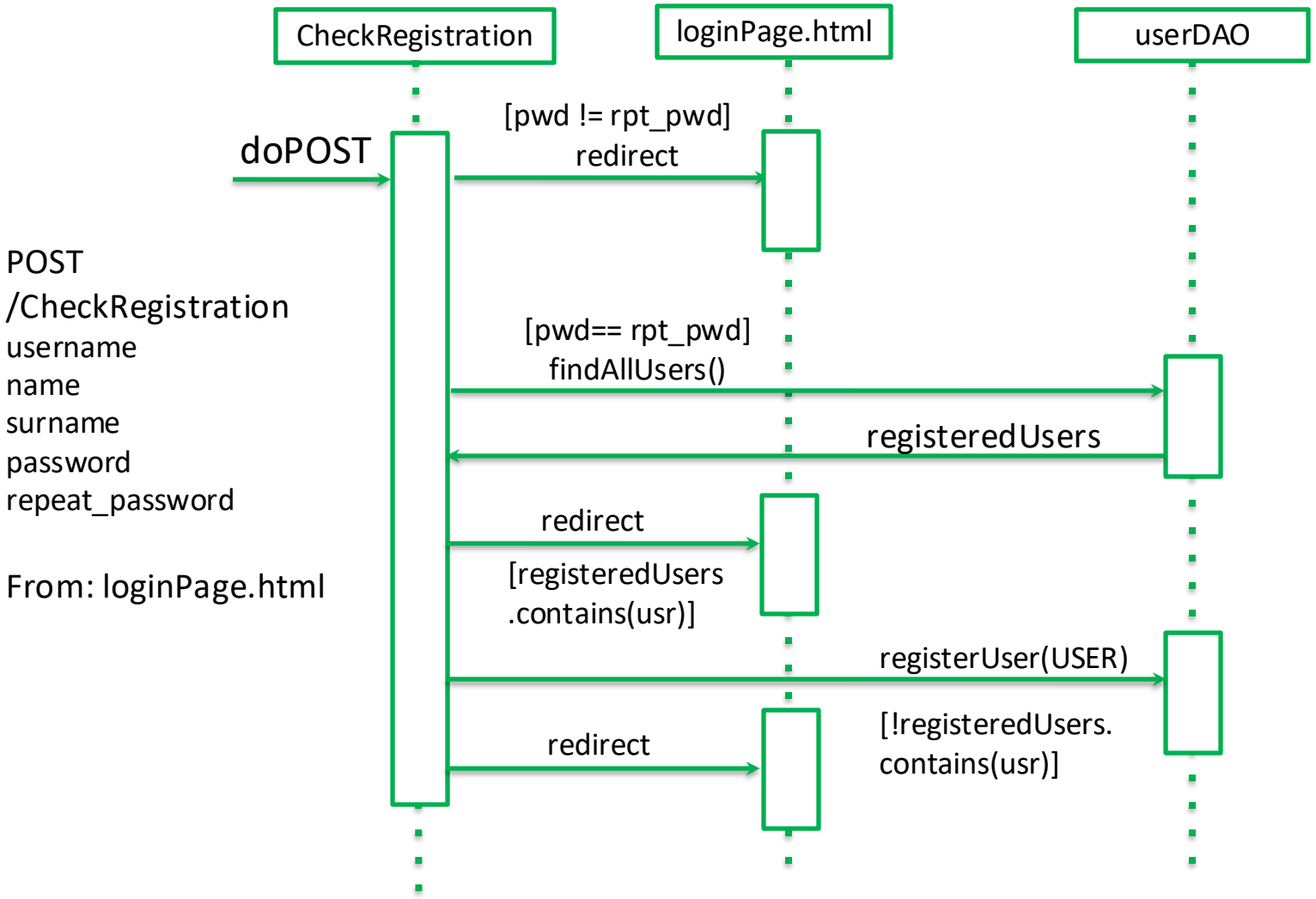
Pure html

Evento: login



Pure html

Evento: registrazione utente

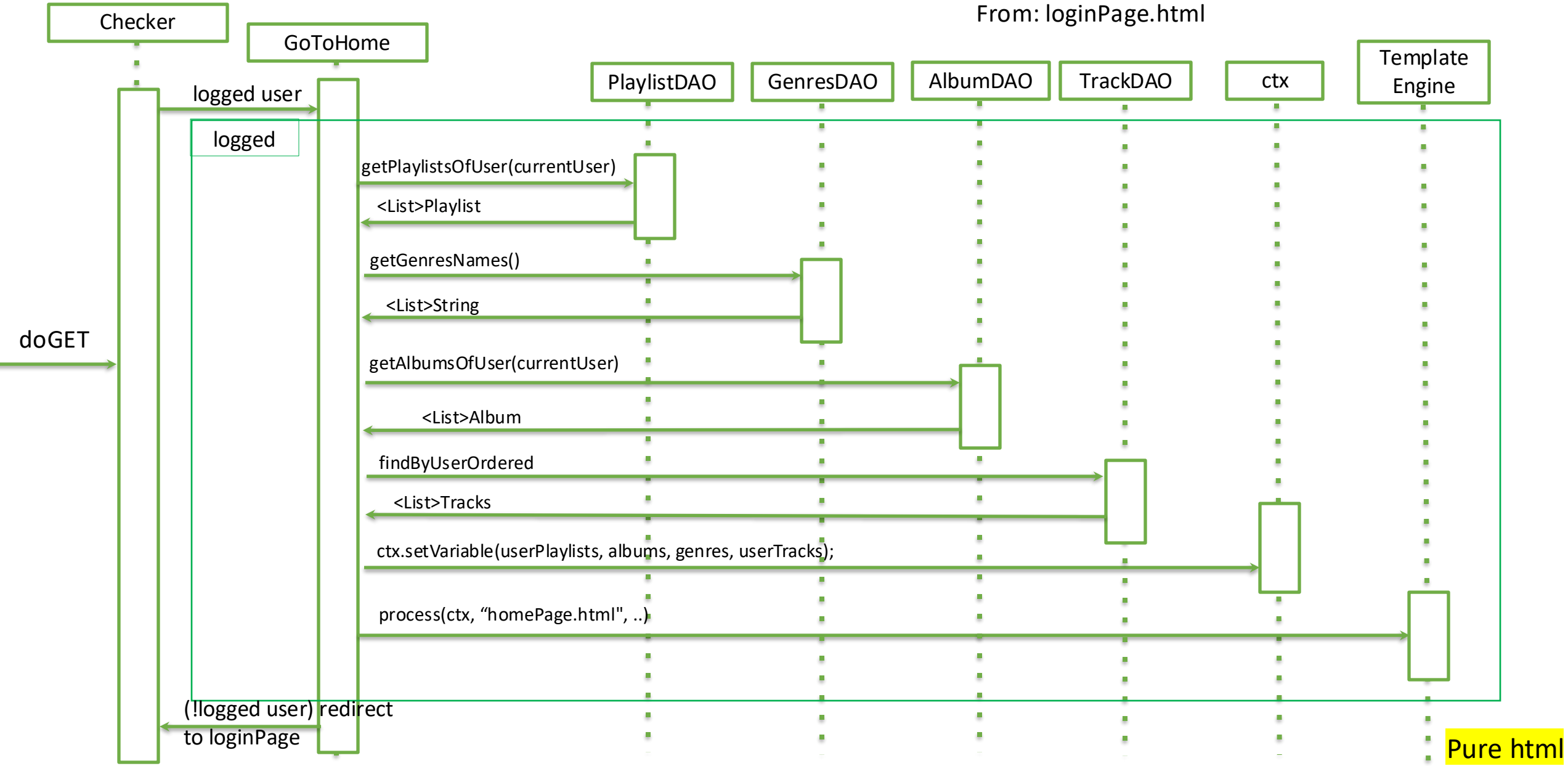


Pure html

Evento: caricamento homepage

GET
/GoToHome

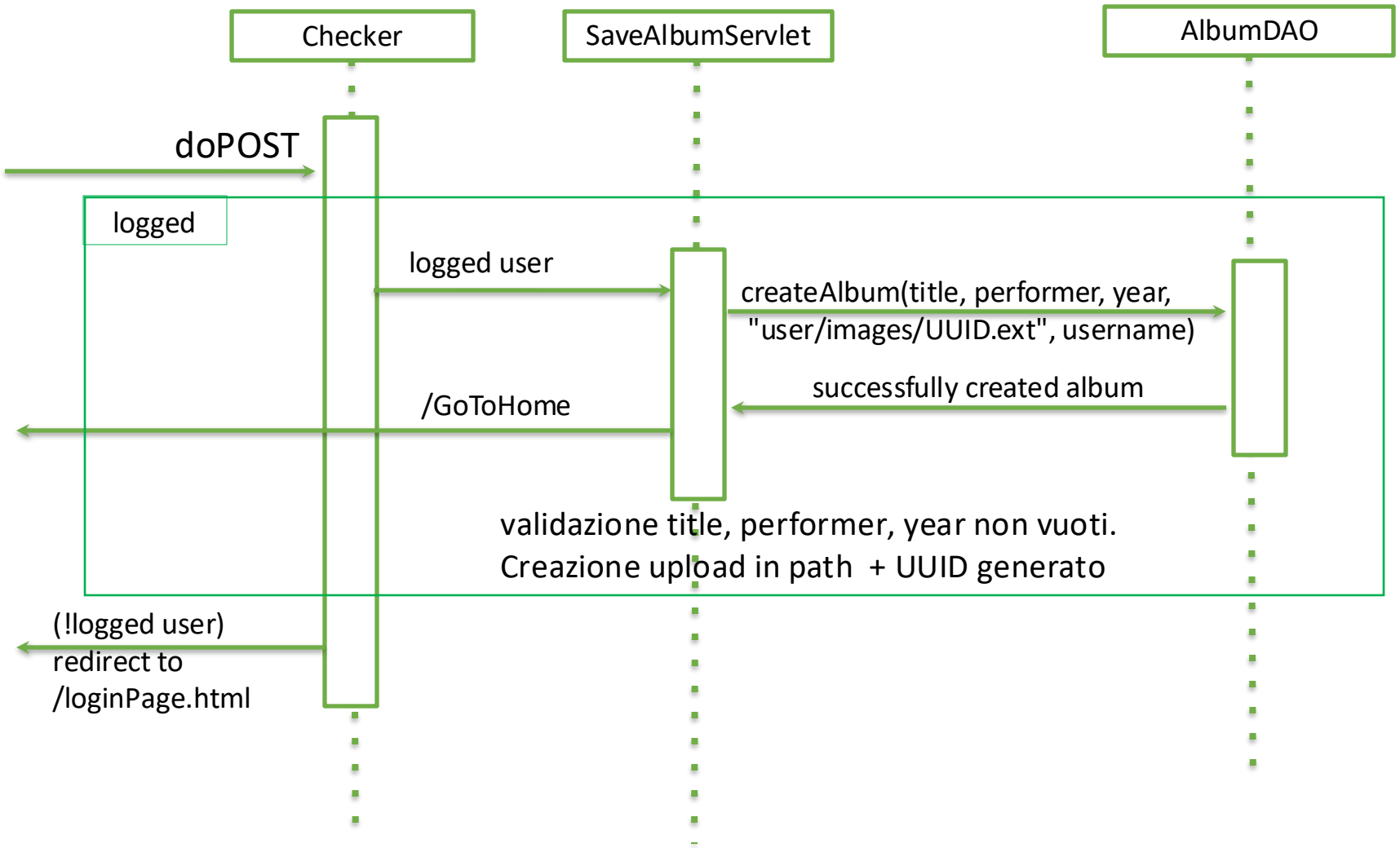
From: loginPage.html



Evento: salvataggio di un album

POST
/SaveAlbum
title
performer
year
imagePart

From:
homePage.html



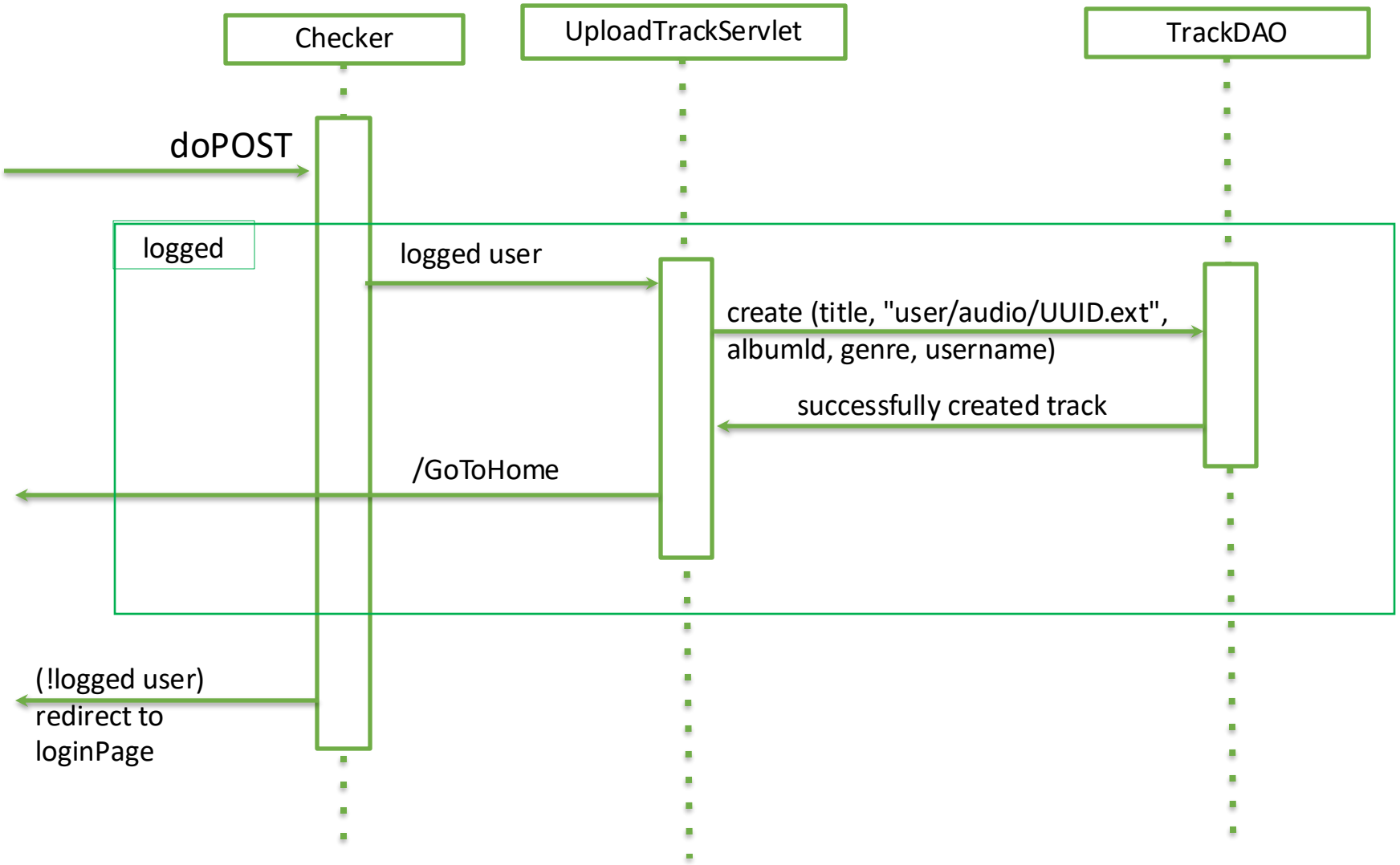
In caso di errori nel riempimento del form da parte dell'utente, viene mostrato un messaggio di errore

Pure html

Evento: salvataggio di una traccia

POST
/UploadTrack
albumId
title
genre
filePart

From:
homePage.html



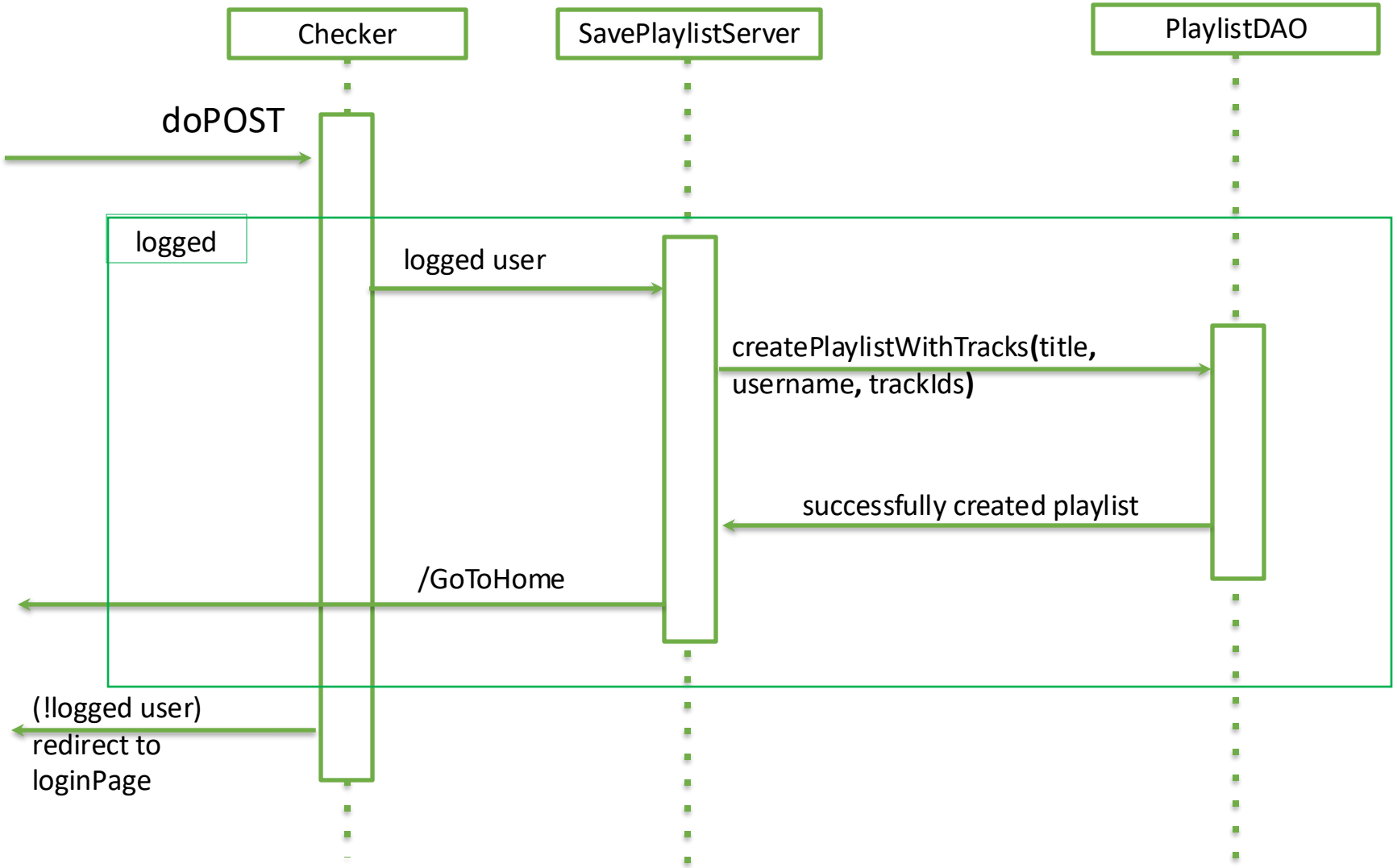
In caso di errori nel riempimento del form da parte dell'utente, viene mostrato un messaggio di errore

Pure html

Evento: creazione di una playlist

POST
/SavePlaylist
Title
TrackIds[ ]

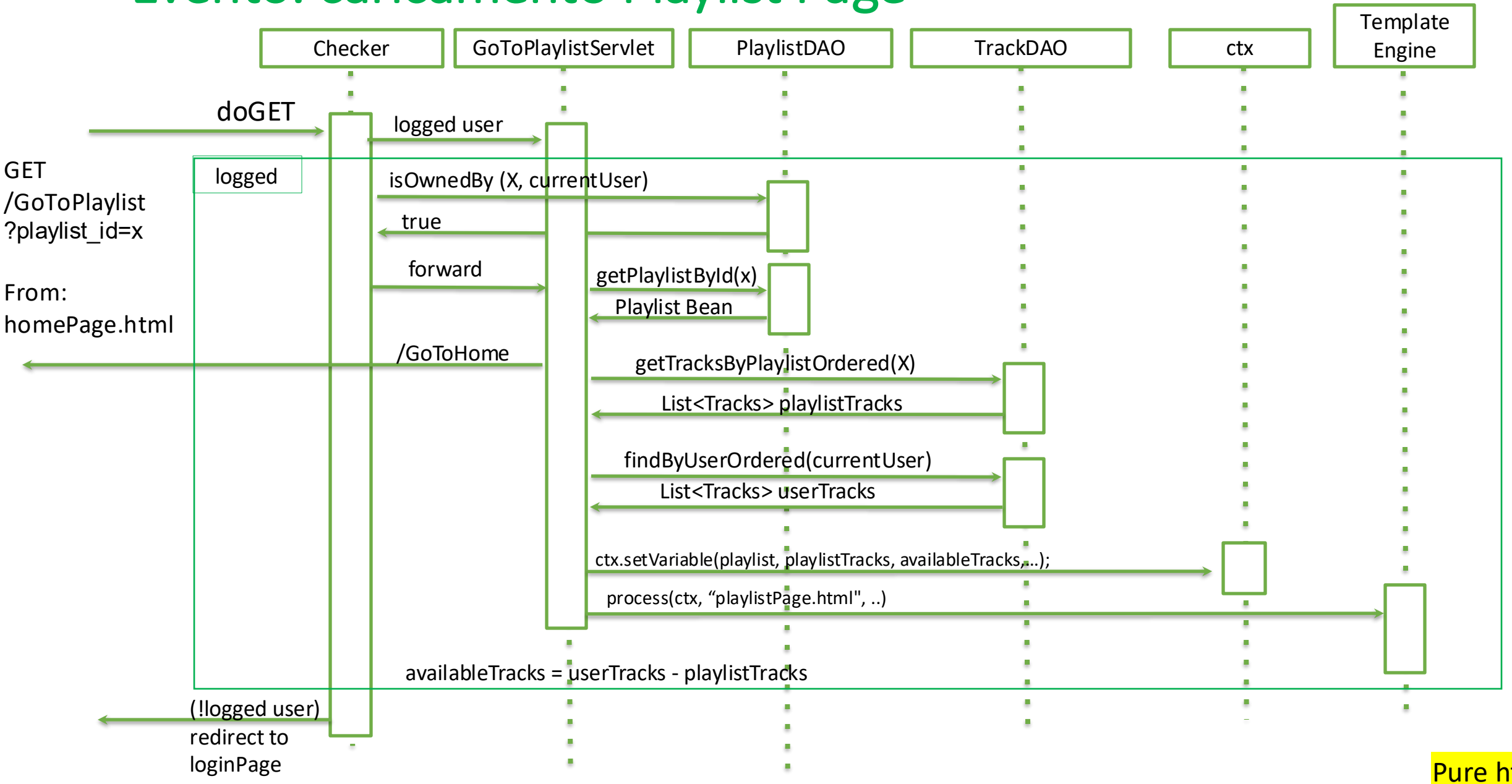
From:
homePage.html



La sanificazione dei parametri con relativa gestione errori è stata omessa

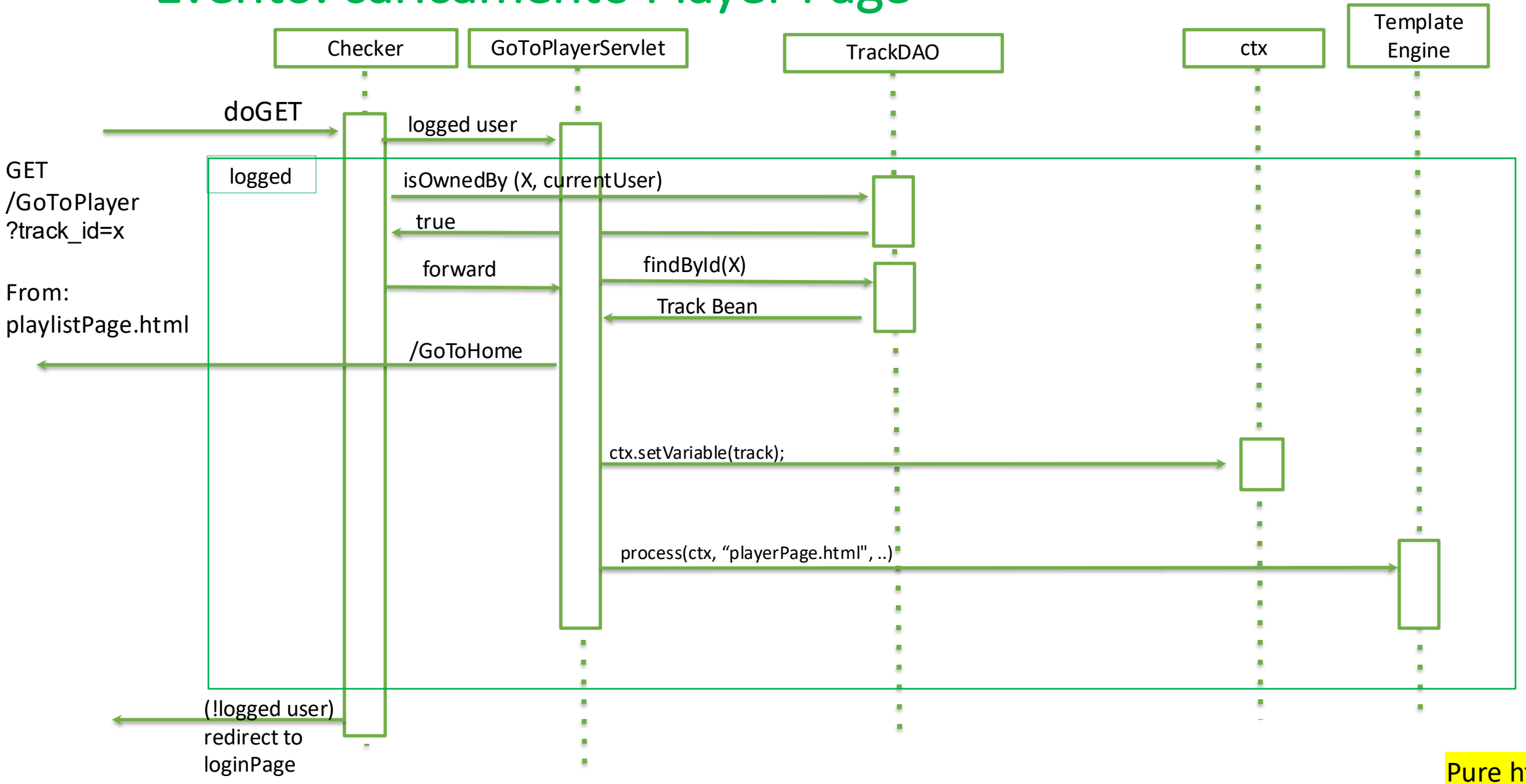
Pure html

Evento: caricamento Playlist Page



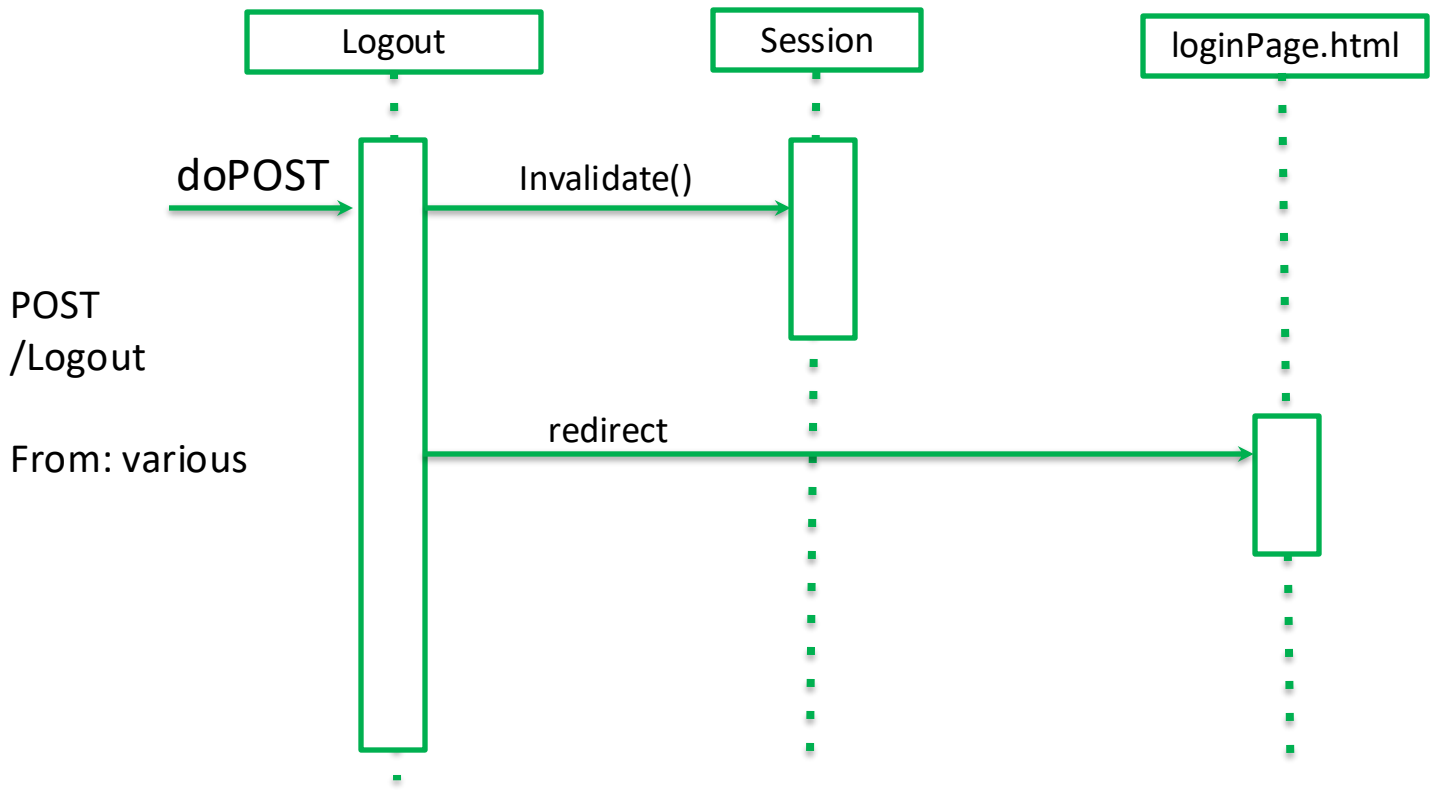
Pure html

Evento: caricamento Player Page



Pure html

Evento: logout



Pure html

PAGINE/FINESTRE, COMPONENTI VISIVI, EVENTI, AZIONI

Specifica: versione JavaScript

Si realizzi un'applicazione client server web che modifica le specifiche precedenti come segue:

- dopo il login dell'utente, l'intera applicazione è realizzata con un'unica pagina.
- Ogni interazione dell'utente è gestita senza ricaricare completamente la pagina, ma produce l'invocazione asincrona del server e l'eventuale modifica del contenuto da aggiornare a seguito dell'evento.
- L'evento di visualizzazione del blocco precedente/successivo è gestito a lato client senza generare una richiesta al server.
- L'applicazione deve consentire all'utente di riordinare le playlist con un criterio personalizzato diverso da quello di default. Dalla HOME, con un link associato a ogni playlist, si accede a una finestra modale RIORDINO, che mostra la lista completa dei brani della playlist ordinati secondo il criterio corrente (personalizzato o di default). L'utente può trascinare il titolo di un brano nell'elenco e collocarlo in una posizione diversa per realizzare l'ordinamento che desidera, senza invocare il server. Quando l'utente ha raggiunto l'ordinamento desiderato, usa un bottone "salva ordinamento", per memorizzare la sequenza sul server. Ai successivi accessi, l'ordinamento personalizzato è usato al posto di quello di default. Un brano aggiunto a una playlist con ordinamento personalizzato è inserito nell'ultima posizione.

Architettura Globale

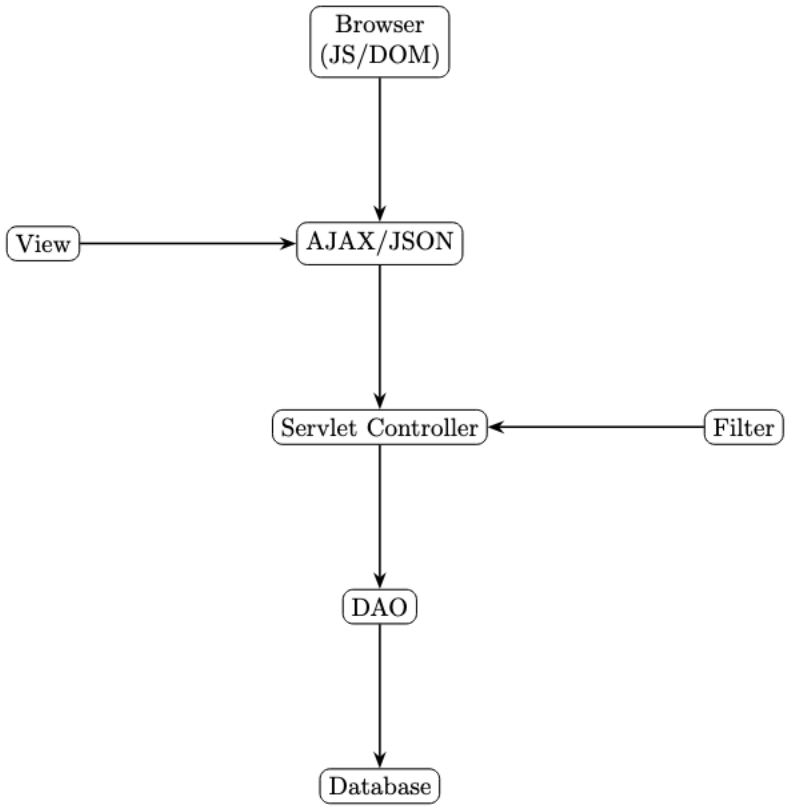
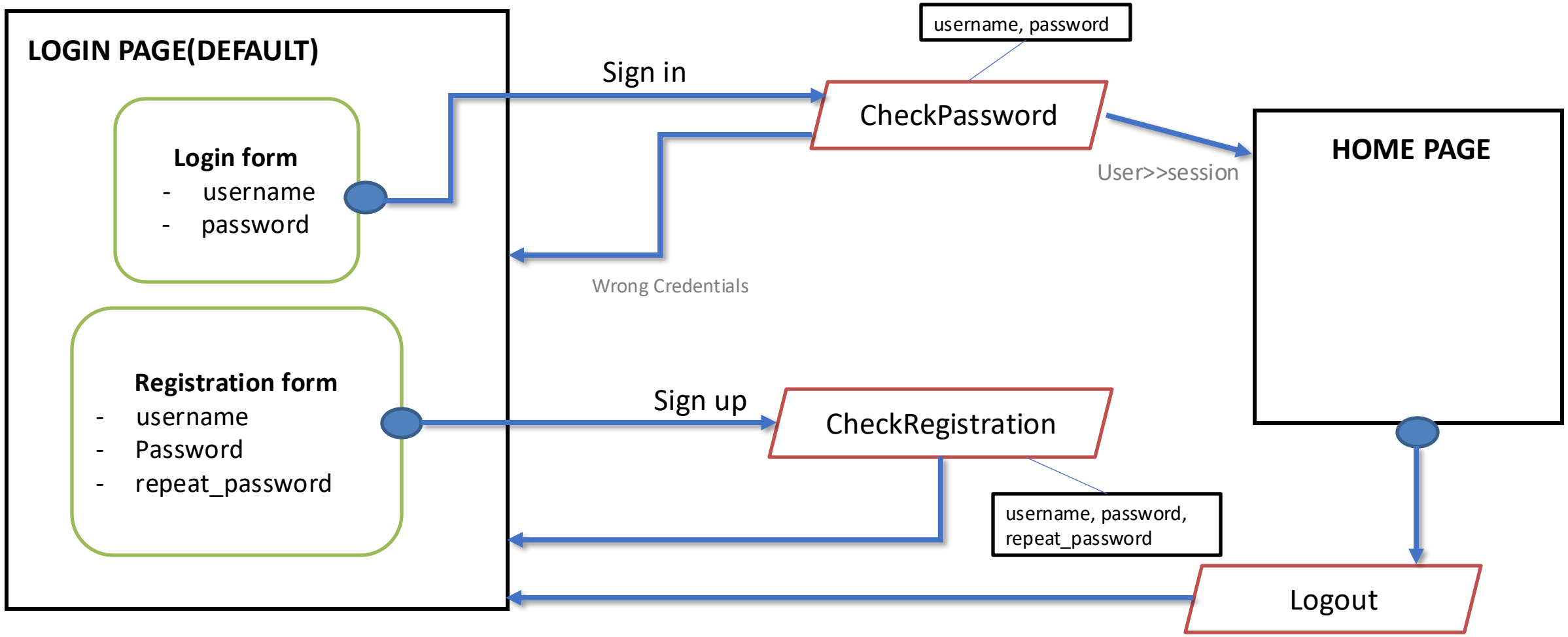


Figure 1: Flusso architetturale dell'applicazione

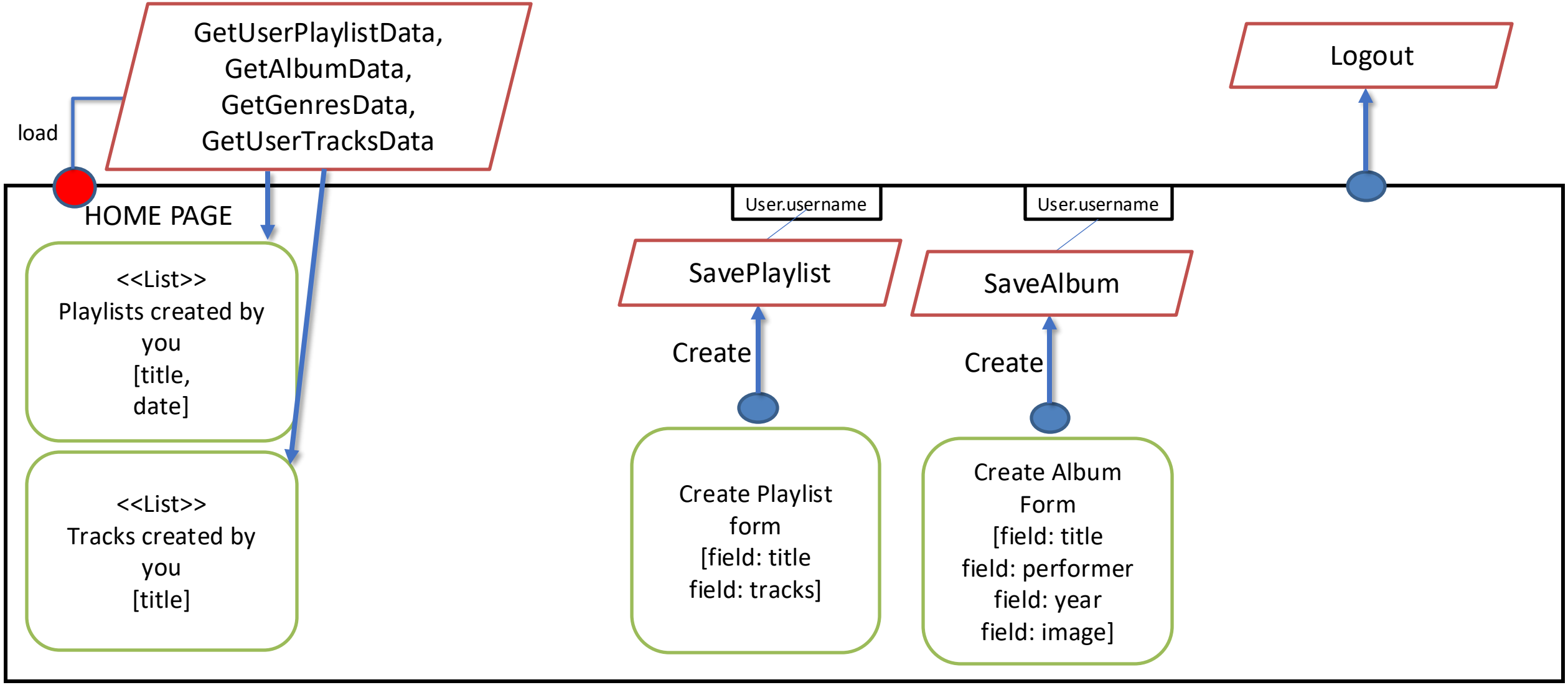
- **Browser (JS/DOM)**
  L'interfaccia lato client, dove il codice JavaScript manipola il DOM.
- **View e Controller Client Side**
  Rappresenta la parte visuale che invia richieste AJAX/JSON al controller.
  - loginPage.html + LoginPageManager.js
  - homePage.html + HomePageManager.js (Page Orchestrator)
- **AJAX/JSON**
  Canale di comunicazione asincrona tra client e server.
- **Servlet Controller (Controller Server Side)**
  Servlet dedicate (CheckPassword, UploadTrack, SavePlaylist …) instradate dal filtro di sicurezza Checker.
- **Filter**
  Componenti che intercettano la richiesta prima che arrivi al controller (es. per autenticazione o logging).
- **DAO (Data Access Object)**
  Livello che incapsula l'accesso ai dati, interfacciante il database.
- **Beans**
- **Database**
  Archivio persistente delle informazioni.

Application design



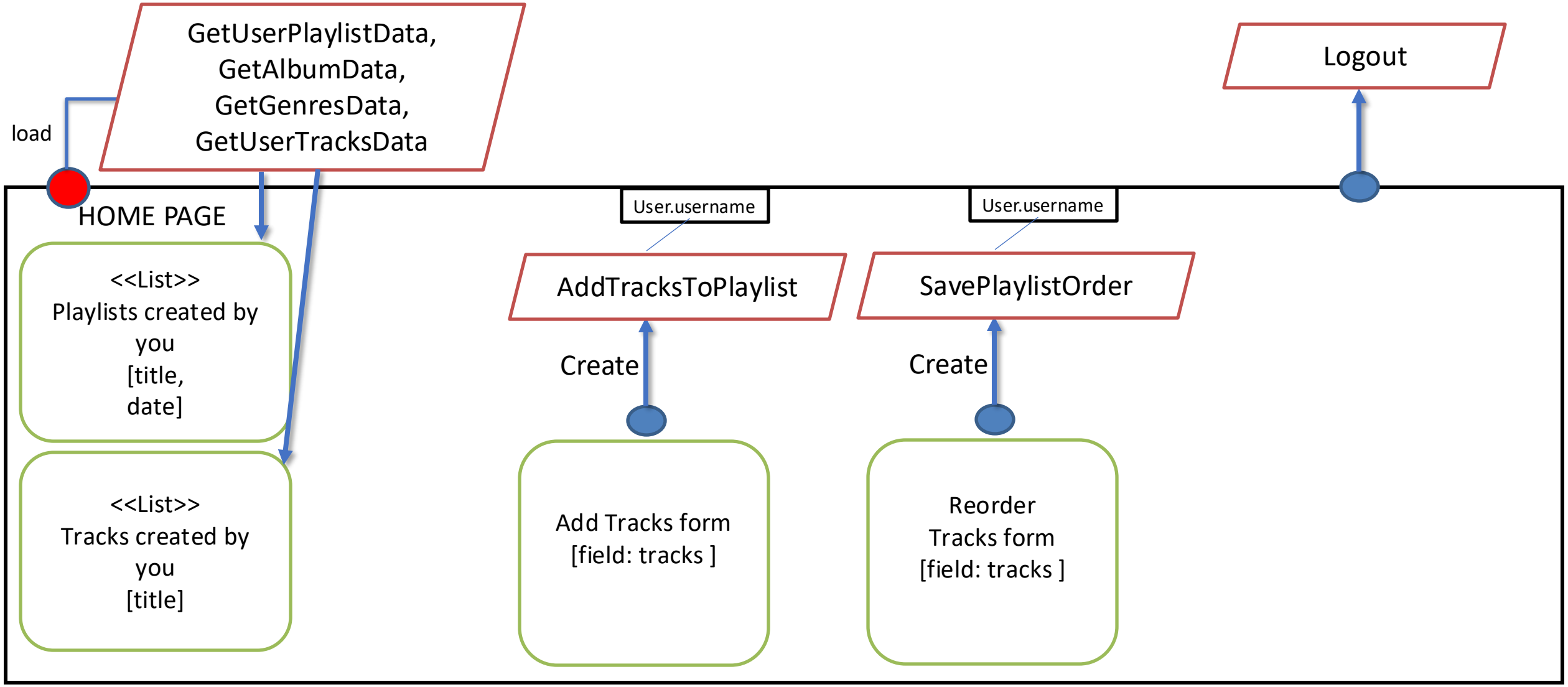
Javascript

Application design



Javascript

Application design



Javascript

Table 1: Moduli lato **client** (browser + JavaScript)

Modulo	**Layer**	**Responsabilità principale / API pubbliche**
`Utils.js`	Client	Wrapper AJAX `makeCall()`, reset form, gestione errori HTTP.
`LoginPageManager.js`	Client	Form login/registrazione; `registerEvents()`.
`HomePageManager.js`	Client	Page Orchestrator: `start()`, `refresh()`, coordinamento componenti.
`PlaylistTable.js`	Client	Tabella playlist; `show()`, `reset()`, apertura dettaglio e modale riordino.
`AlbumCreator.js`	Client	Form "Create Album"; validazione immagine ($\leq$ 5 MB); `registerEvents()`, `show()`.
`TrackUploader.js`	Client	Form upload brano; preload album/genere; file audio $\leq$ 10 MB.
`PlaylistCreator.js`	Client	Form creazione playlist; checkbox brani; validazione ($\geq$ 1 brano).
`PlaylistDetailView.js`	Client	Griglia 1×5 brani, paginazione client, form aggiunta brani.
`PlayerView.js`	Client	Recupera dati brano (`GetTrackData`); costruisce player audio.
`ReorderModal.js`	Client	Modale drag&drop per ordinamento brani; `open()`, `saveOrder()`.

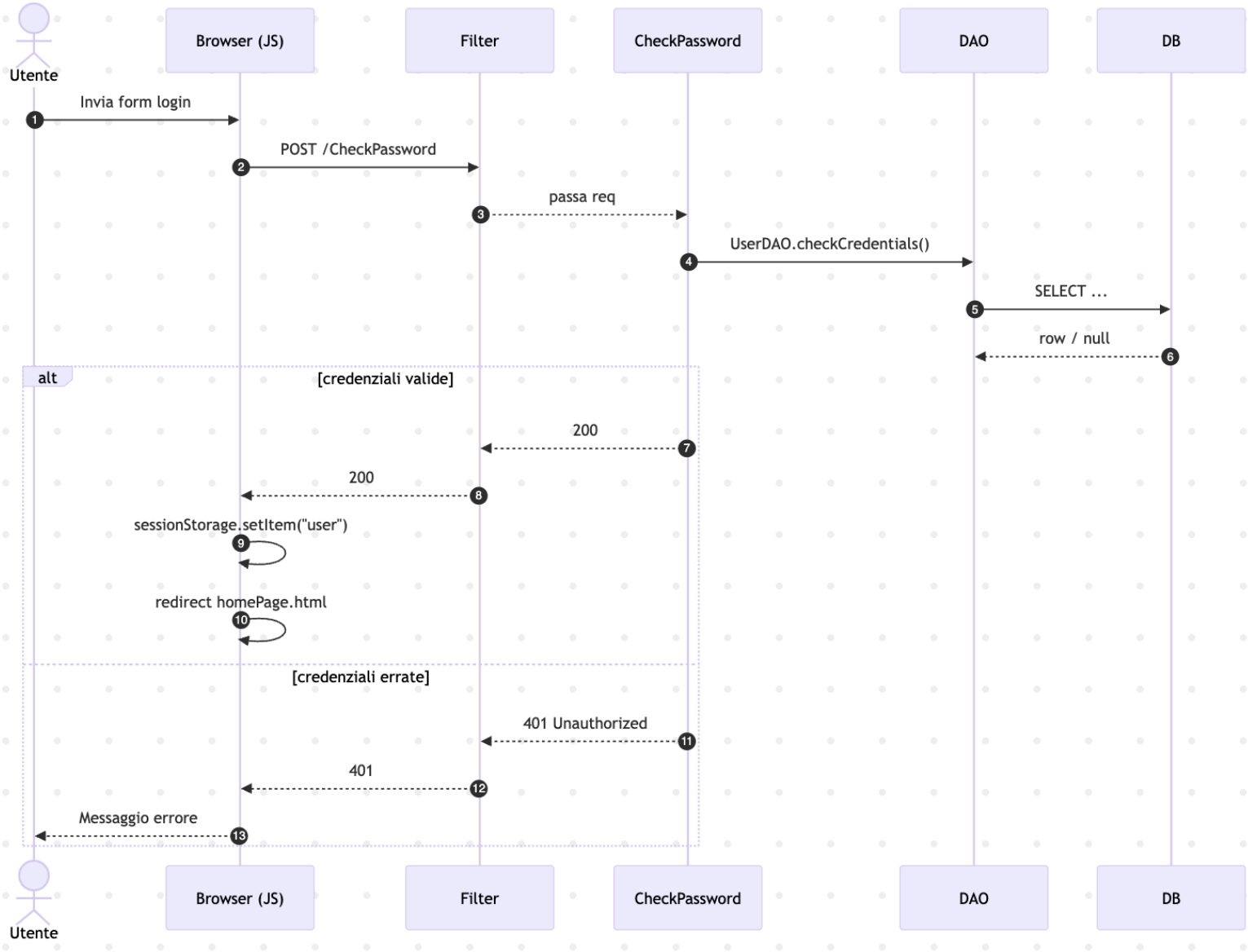
Javascript

Table 2: Moduli lato **server** (Java EE)

Classe / Servlet	**Layer**	**Responsabilità principale**
`Checker`	Filter	Verifica sessione, proprietà risorse, header no-cache.
`ConnectionHandler`	Utility	Fornisce connessioni JDBC (factory).
Servlet di autenticazione		
`CheckPassword`	Controller	Login: valida credenziali, crea sessione.
`CheckRegistration`	Controller	Registrazione utente, gestione duplicati.
`Logout`	Controller	Invalida sessione, redirect a login.
Servlet CRUD Album / Track		
`SaveAlbum`	Controller	Crea nuovo album.
`UploadTrack`	Controller	Upload file audio + INSERT Track.
`GetAlbumData`	Controller	Restituisce lista album (JSON).
`GetGenresData`	Controller	Restituisce lista generi (JSON).
`GetTrackData`	Controller	Dettaglio singolo brano (JSON).
Servlet CRUD Playlist		
`SavePlaylist`	Controller	Crea playlist + INSERT brani scelti.
`AddTracksToPlaylist`	Controller	Aggiunge brani esistenti a playlist.
`SavePlaylistOrder`	Controller	Salva ordinamento personalizzato.
`GetPlaylistData`	Controller	Playlist + brani (JSON).
`GetUserPlaylistsData`	Controller	Lista playlist utente (JSON).
DAO (Data Access Object)		
`UserDAO`, `AlbumDAO`, `TrackDAO`, `PlaylistDAO`, `GenresDAO`	DAO	Query CRUD, metodi `isOwnedBy()`, transazioni.

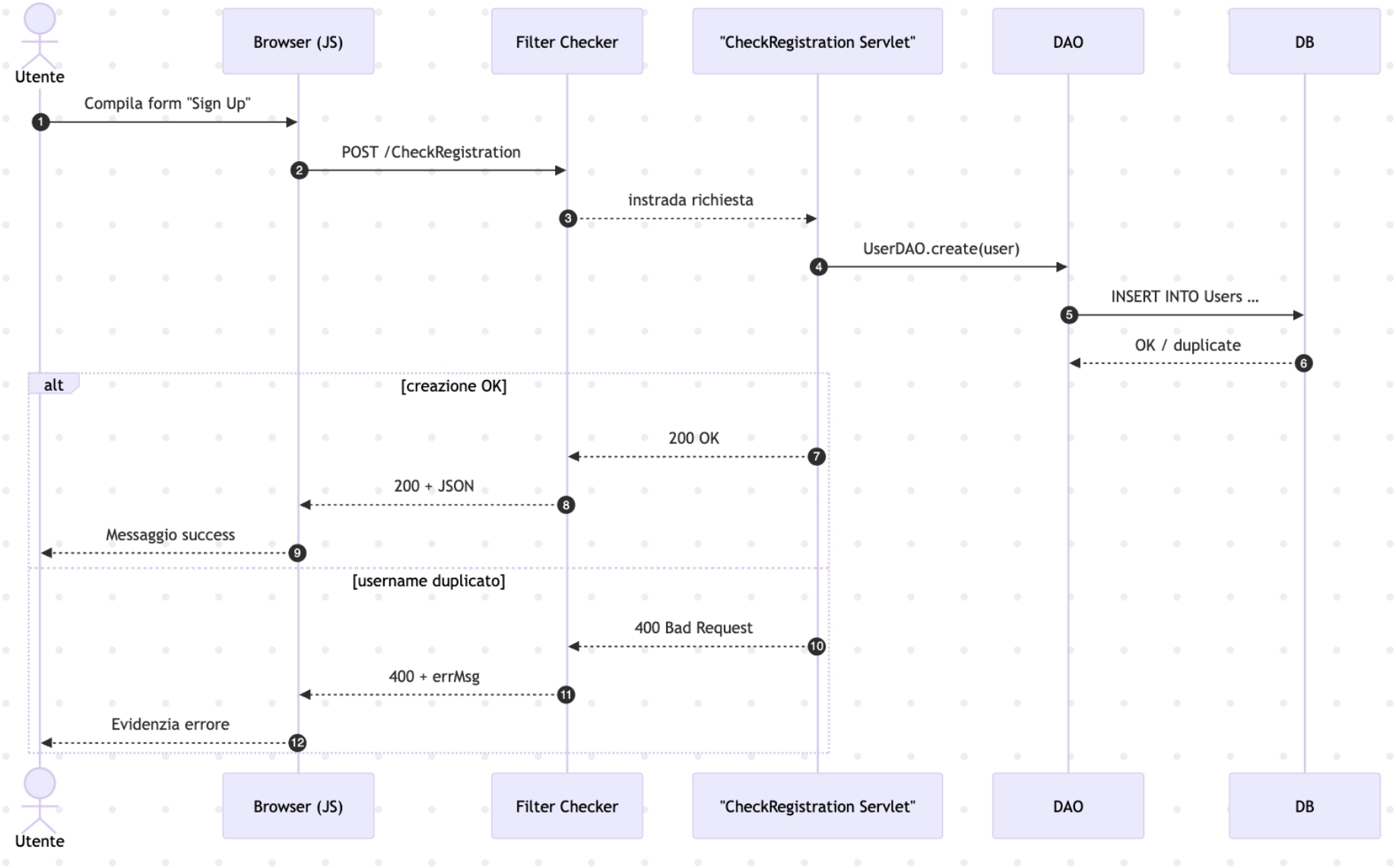
Javascript

Evento: Login



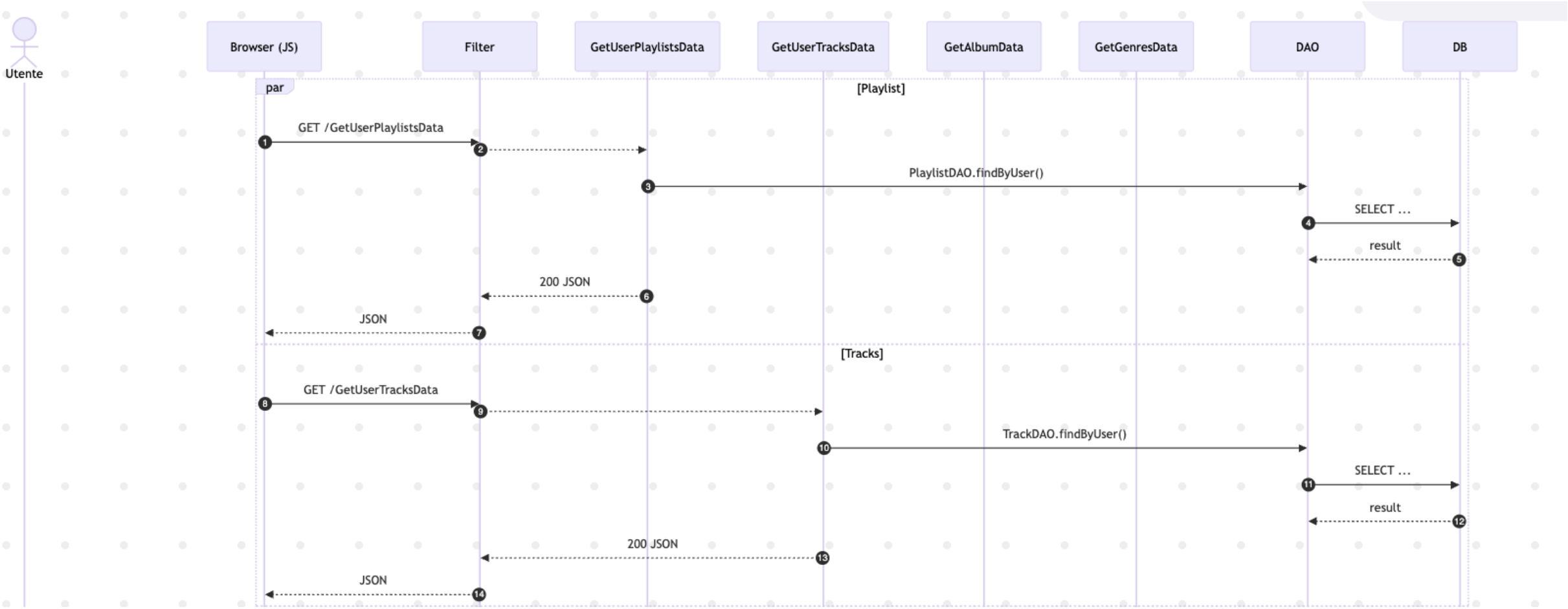
Javascript

Evento: Registrazione Utente



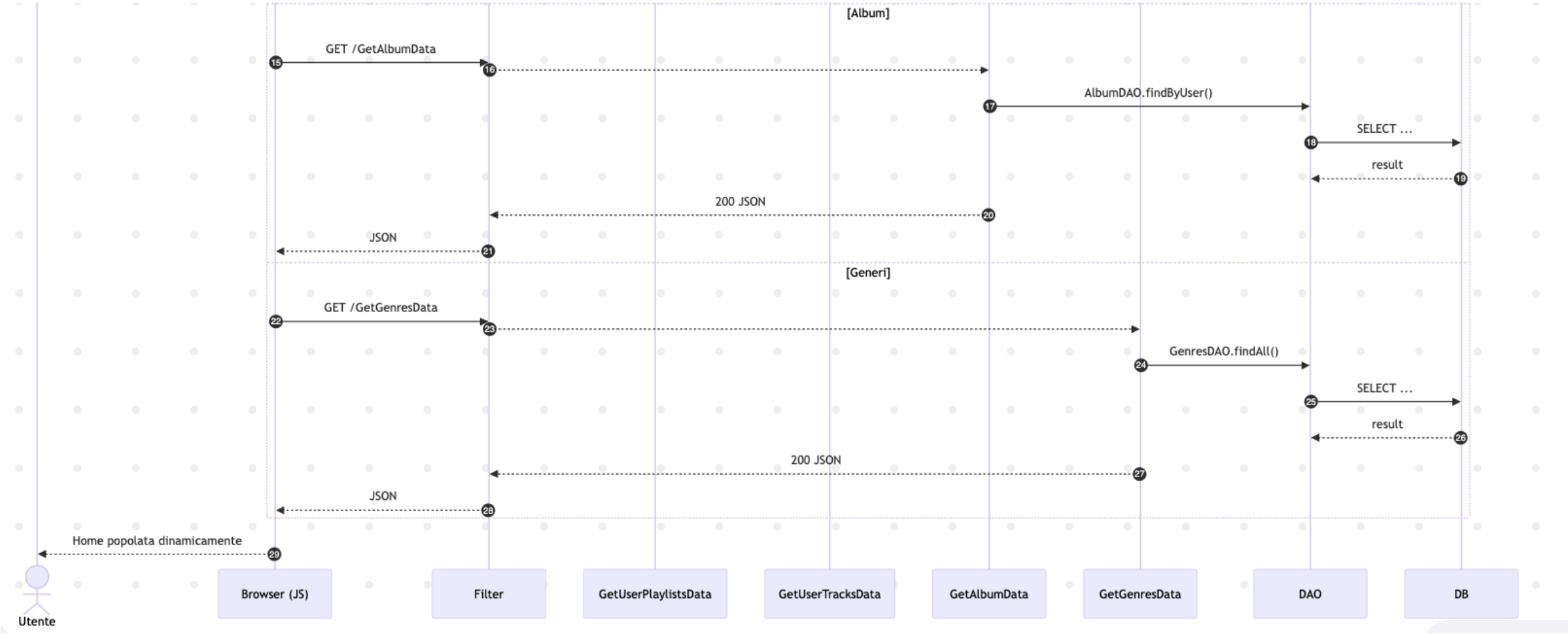
Javascript

Evento: caricamento Home Page -1



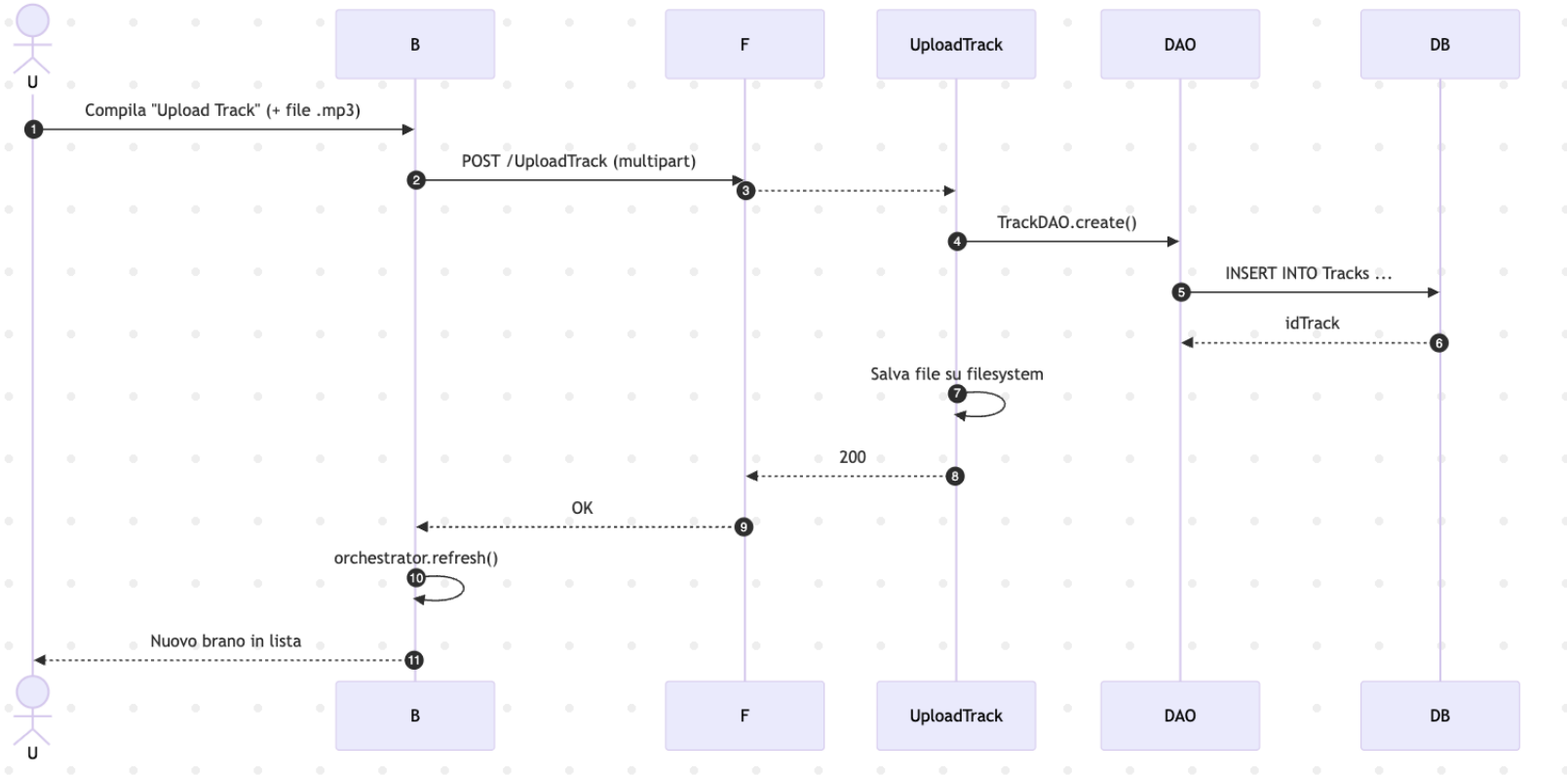
Javascript

Evento: caricamento Home Page -2



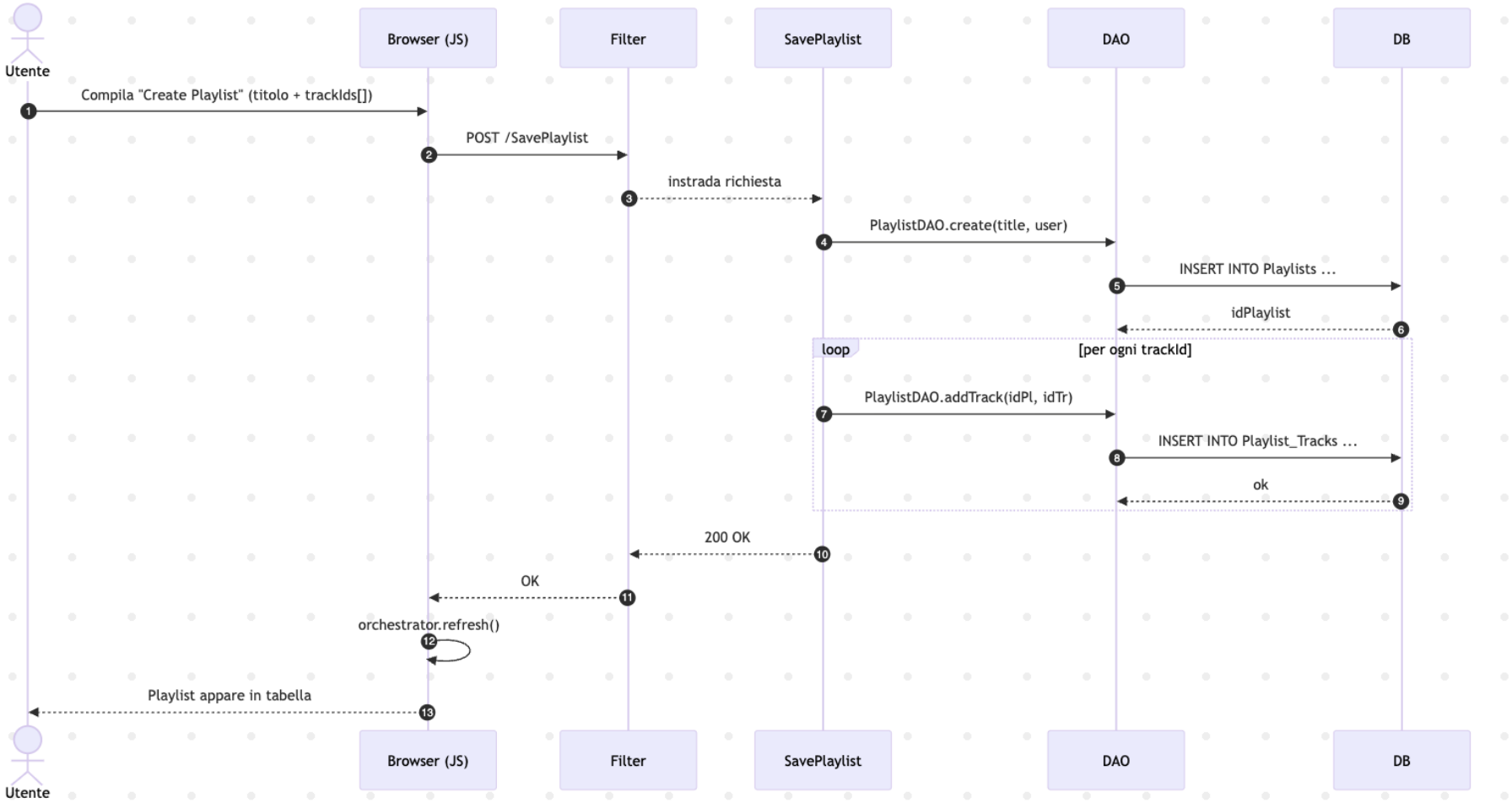
Javascript

Evento: caricamento Traccia



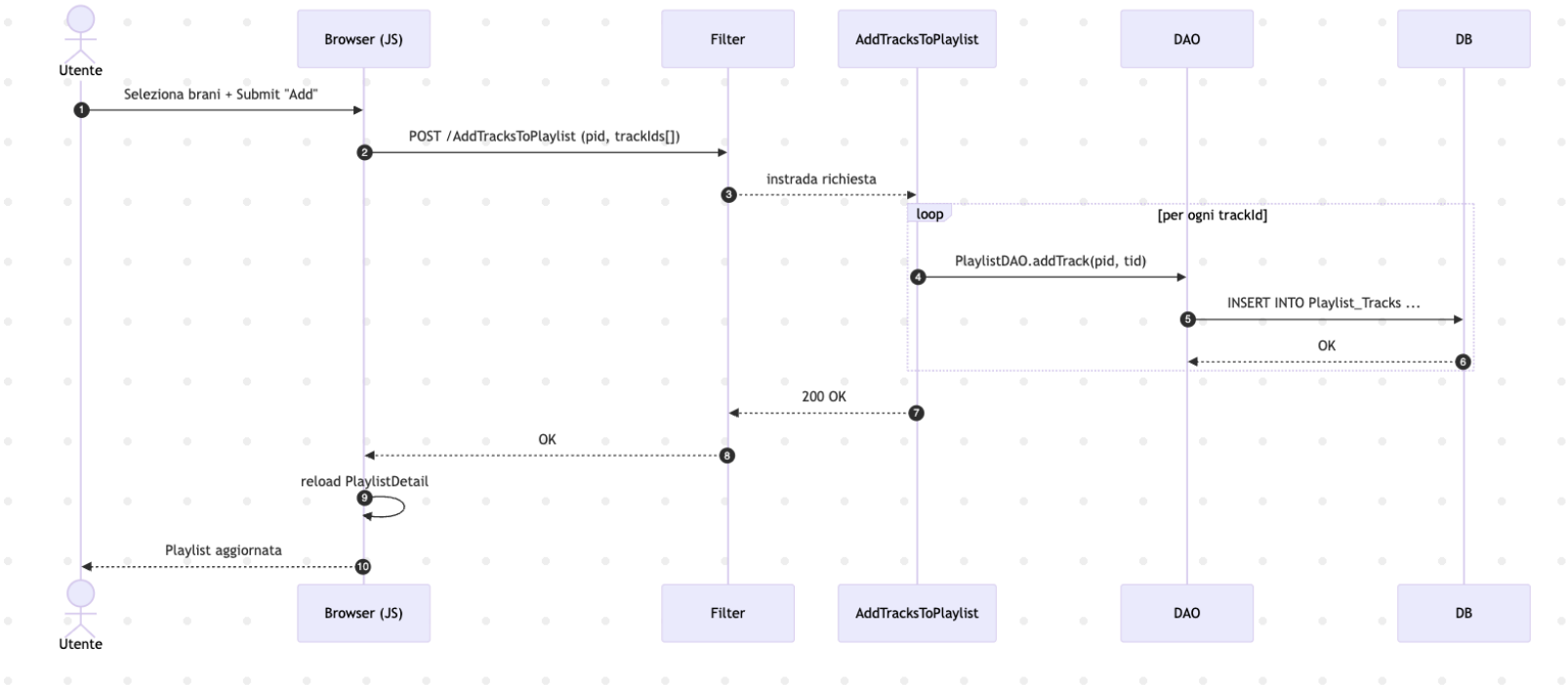
Javascript

Evento: caricamento Playlist



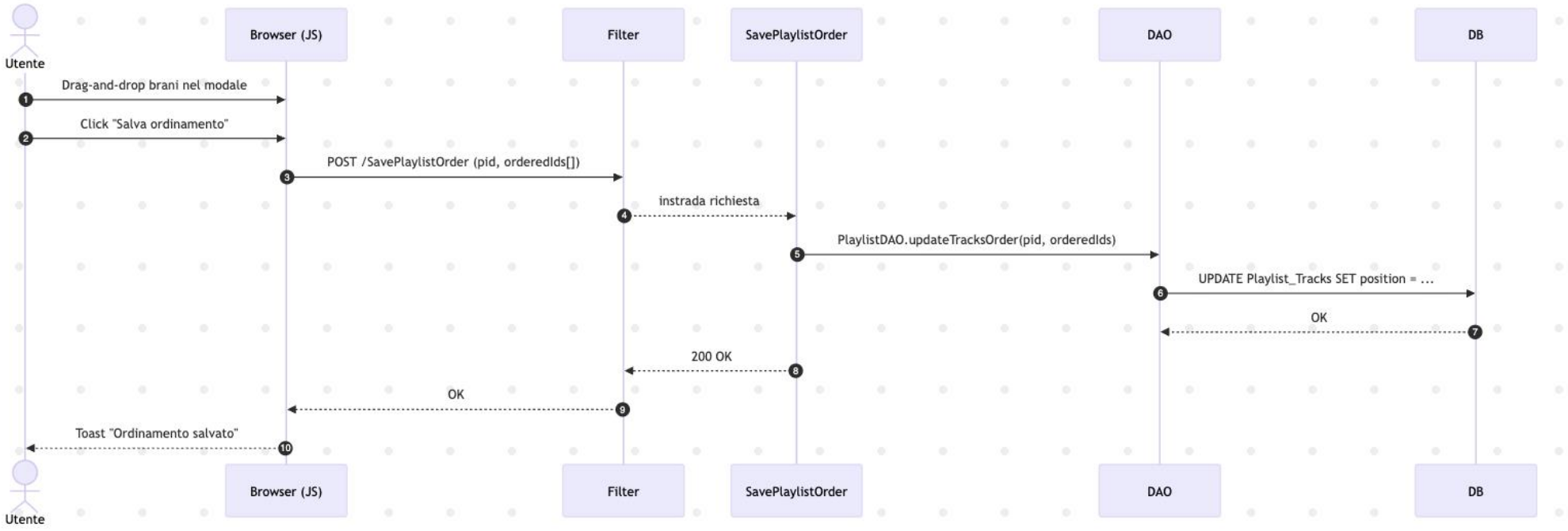
Javascript

Evento: aggiunta tracce ad una playlist



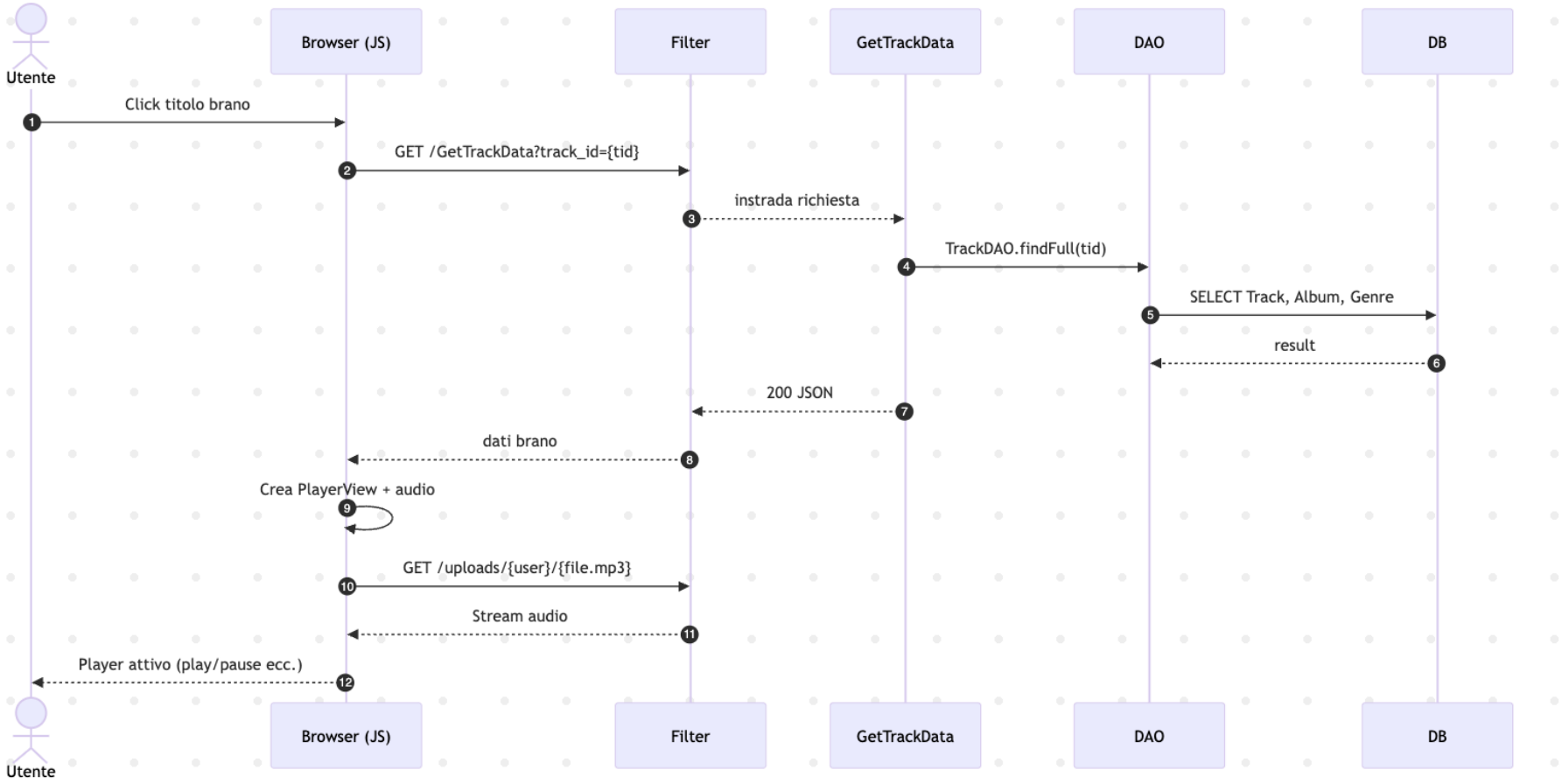
Javascript

Evento: riordino modale



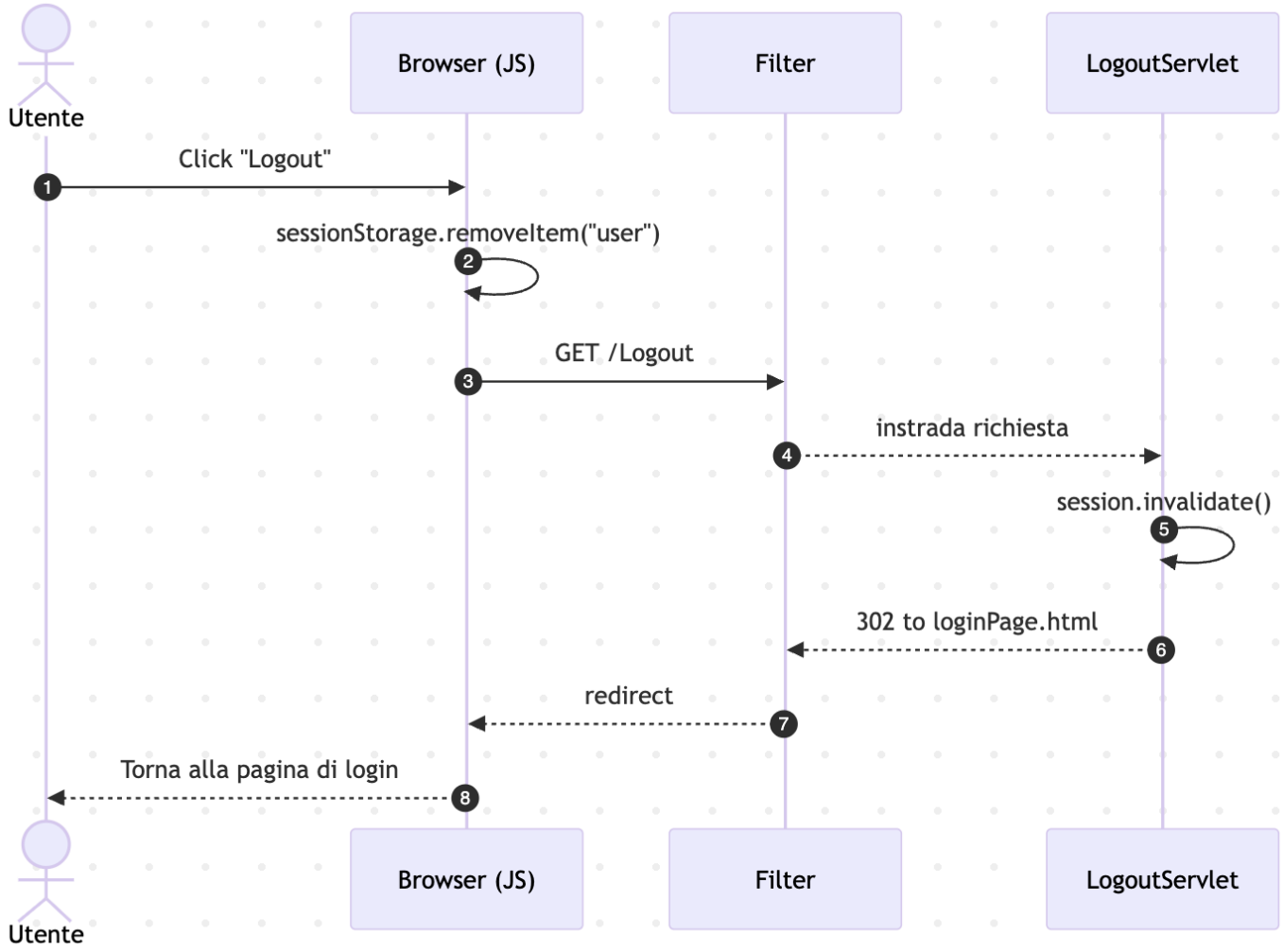
Javascript

Evento: player



Javascript

Evento: Logout



Javascript

Sicurezza e Robustezza

- **Autenticazione** – Checker reindirizza a loginPage.html se la sessione non contiene l'attributo user.
- **Autorizzazione per risorse statiche** (/uploads/{username}/…) – Il filtro blocca (403) richieste con username diverso dal loggato.
- **Proprietà dei dati** – Per servlet GetTrackData, GetPlaylistData e operazioni di mutazione (AddTracksToPlaylist, SavePlaylistOrder) viene verificato che l'ID richiesto appartenga all'utente (metodi TrackDAO.isOwnedBy, PlaylistDAO.isOwnedBy).
- **SQL Injection** – tutte le query nel DAO usano PreparedStatement con segnaposto ?.
- **Limite dimensioni upload** – Annota @MultipartConfig(maxFileSize=10485760) su UploadTrack.
- **Header di cache** – Il filtro aggiunge Cache-Control: no-cache, no-store, must-revalidate per le pagine protette.
- **Gestione delle transazioni** – *connection.commit()* e *connection.rollback()* e *connection.autocommit(false)*;

Javascript